

Problem Statement

Business Context

A sales forecast is a prediction of future sales revenue based on historical data, industry trends, and the status of the current sales pipeline. Businesses use the sales forecast to estimate weekly, monthly, quarterly, and annual sales totals. A company needs to make an accurate sales forecast as it adds value across an organization and helps the different verticals to chalk out their future course of action.

Forecasting helps an organization plan its sales operations by region and provides valuable insights to the supply chain team regarding the procurement of goods and materials. An accurate sales forecast process has many benefits which include improved decision-making about the future and reduction of sales pipeline and forecast risks. Moreover, it helps to reduce the time spent in planning territory coverage and establish benchmarks that can be used to assess trends in the future.

Objective

SuperKart is a retail chain operating supermarkets and food marts across various tier cities, offering a wide range of products. To optimize its inventory management and make informed decisions around regional sales strategies, SuperKart wants to accurately forecast the sales revenue of its outlets for the upcoming quarter.

To operationalize these insights at scale, the company has partnered with a data science firm—not just to build a predictive model based on historical sales data, but to develop and deploy a robust forecasting solution that can be integrated into SuperKart's decision-making systems and used across its network of stores.

Data Description

The data contains the different attributes of the various products and stores. The detailed data dictionary is given below.

- **Product_Id** - unique identifier of each product, each identifier having two letters at the beginning followed by a number.
- **Product_Weight** - weight of each product
- **Product_Sugar_Content** - sugar content of each product like low sugar, regular and no sugar
- **Product_Allocated_Area** - ratio of the allocated display area of each product to the total display area of all the products in a store
- **Product_Type** - broad category for each product like meat, snack foods, hard drinks, dairy, canned, soft drinks, health and hygiene, baking goods, bread, breakfast, frozen foods, fruits

and vegetables, household, seafood, starchy foods, others

- **Product_MRP** - maximum retail price of each product
- **Store_Id** - unique identifier of each store
- **Store_Establishment_Year** - year in which the store was established
- **Store_Size** - size of the store depending on sq. feet like high, medium and low
- **Store_Location_City_Type** - type of city in which the store is located like Tier 1, Tier 2 and Tier 3. Tier 1 consists of cities where the standard of living is comparatively higher than its Tier 2 and Tier 3 counterparts.
- **Store_Type** - type of store depending on the products that are being sold there like Departmental Store, Supermarket Type 1, Supermarket Type 2 and Food Mart
- **Product_Store_Sales_Total** - total revenue generated by the sale of that particular product in that particular store

Installing and Importing the necessary libraries

```
#Installing the libraries with the specified versions
```

```
!pip install numpy==2.0.2 pandas==2.2.2 scikit-learn==1.6.1 matplotlib==3.10.0 seaborn==0.13.2
```

Note:

- After running the above cell, kindly restart the notebook kernel (for Jupyter Notebook) or runtime (for Google Colab) and run all cells sequentially from the next cell.
- On executing the above line of code, you might see a warning regarding package dependencies. This error message can be ignored as the above code ensures that all necessary libraries and their dependencies are maintained to successfully execute the code in this notebook.

```
import warnings
warnings.filterwarnings("ignore")

# Libraries to help with reading and manipulating data
import numpy as np
import pandas as pd

# For splitting the dataset
from sklearn.model_selection import train_test_split

# Libraries to help with data visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Removes the limit for the number of displayed columns
pd.set_option("display.max_columns", None)
# Sets the limit for the number of displayed rows
pd.set_option("display.max_rows", 100)

# Libraries different ensemble classifiers
from sklearn.ensemble import (
```

```

    BaggingRegressor,
    RandomForestRegressor,
    AdaBoostRegressor,
    GradientBoostingRegressor,
)
from xgboost import XGBRegressor
from sklearn.tree import DecisionTreeRegressor

# Libraries to get different metric scores
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    mean_squared_error,
    mean_absolute_error,
    r2_score,
    mean_absolute_percentage_error
)

# To create the pipeline
from sklearn.compose import make_column_transformer
from sklearn.pipeline import make_pipeline, Pipeline

# To tune different models and standardize
from sklearn.model_selection import GridSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# To serialize the model
import joblib

# os related functionalities
import os

# API request
import requests

# for hugging face space authentication to upload files
from huggingface_hub import login, HfApi

```

Loading the dataset

```
kart = pd.read_csv("SuperKart.csv") #Complete the code to read the data
```

```
# copying data to another variable to avoid any changes to original data
data = kart.copy()
```

```
# Observation : The data is copied into a new variable to avoid getting corrupted.
```

Data Overview

View the first and last 5 rows of the dataset

```
data.head()
```

	Product_Id	Product_Weight	Product_Sugar_Content	Product_Allocated_Area	Product_Type
0	FD6114	12.66	Low Sugar	0.027	Frozen Foooc
1	FD7839	16.54	Low Sugar	0.144	Dair
2	FD5075	14.28	Regular	0.031	Canne
3	FD8233	12.10	Low Sugar	0.112	Baking Goooc
4	NC1180	9.57	No Sugar	0.010	Health an Hygier

Observation : The first 5 records of the data are printed. there are no null values

```
data.tail()
```

	Product_Id	Product_Weight	Product_Sugar_Content	Product_Allocated_Area	Product_Type
8758	NC7546	14.80	No Sugar	0.016	Healt Hy
8759	NC584	14.06	No Sugar	0.142	Hous
8760	NC2471	13.48	No Sugar	0.017	Healt Hy
8761	NC7187	13.89	No Sugar	0.193	Hous
8762	FD306	14.73	Low Sugar	0.177	Snack I

Observation : The lasr 5 records of the data are printed. there are no null values

Understand the shape of the dataset

```
print(f"There are {data.shape[0]} rows and {data.shape[1]} columns.")
```

There are 8763 rows and 12 columns.

Observation : There are 8763 rows and 12 columns in the dataset.

Check the data types of the columns for the dataset

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8763 entries, 0 to 8762
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Product_Id                            8763 non-null   object
1   Product_Weight                        8763 non-null   float64
2   Product_Sugar_Content                 8763 non-null   object
3   Product_Allocated_Area                8763 non-null   float64
4   Product_Type                          8763 non-null   object
5   Product_MRP                          8763 non-null   float64
6   Store_Id                             8763 non-null   object
7   Store_Establishment_Year              8763 non-null   int64
8   Store_Size                           8763 non-null   object
9   Store_Location_City_Type              8763 non-null   object
10  Store_Type                           8763 non-null   object
11  Product_Store_Sales_Total             8763 non-null   float64
dtypes: float64(4), int64(1), object(7)
memory usage: 821.7+ KB
```

Observation : There are 12 columns with 7 of type object , 4 of type float and 1

Statistical summary of the data

Let's check the statistical summary of the data.

```
data.describe(include="all").T
```

	count	unique	top	freq	mean	std	r
Product_Id	8763	8763	FD306	1	NaN	NaN	N
Product_Weight	8763.0	NaN	NaN	NaN	12.653792	2.21732	
Product_Sugar_Content	8763	4	Low Sugar	4885	NaN	NaN	N
Product_Allocated_Area	8763.0	NaN	NaN	NaN	0.068786	0.048204	0.0
Product_Type	8763	16	Fruits and Vegetables	1249	NaN	NaN	N
Product_MRP	8763.0	NaN	NaN	NaN	147.032539	30.69411	3
Store_Id	8763	4	OUT004	4676	NaN	NaN	N
Store_Establishment_Year	8763.0	NaN	NaN	NaN	2002.032751	8.388381	198
Store_Size	8763	3	Medium	6025	NaN	NaN	N
Store_Location_City_Type	8763	3	Tier 2	6262	NaN	NaN	N
Store_Type	8763	4	Supermarket Type2	4676	NaN	NaN	N
Product_Store_Sales_Total	8763.0	NaN	NaN	NaN	3464.00364	1065.630494	3

Observation :
The dataset contains 8,763 unique products across 4 different store IDs, with OUT
Low Sugar is the most common product attribute, and Fruits and Vegetables is the

```
# There are 4 types of store and there are totally 16 types of product.  
# There are 3 different types of stores most likely small, medium and large and Mec
```

Checking for duplicate values

```
# checking for duplicate values  
data.duplicated().sum()
```

```
np.int64(0)
```

```
# Observation : There are no zero values.
```

Checking for missing values

```
# checking for missing values in the data  
data.isnull().sum()
```

	0
Product_Id	0
Product_Weight	0
Product_Sugar_Content	0
Product_Allocated_Area	0
Product_Type	0
Product_MRP	0
Store_Id	0
Store_Establishment_Year	0
Store_Size	0
Store_Location_City_Type	0
Store_Type	0
Product_Store_Sales_Total	0

dtype: int64

```
# Observation : There are no missing values in any rows or columns
```

Exploratory Data Analysis (EDA)

Univariate Analysis

```
# function to plot a boxplot and a histogram along the same scale.  
  
def histogram_boxplot(data, feature, figsize=(12, 7), kde=False, bins=None):  
    .....
```

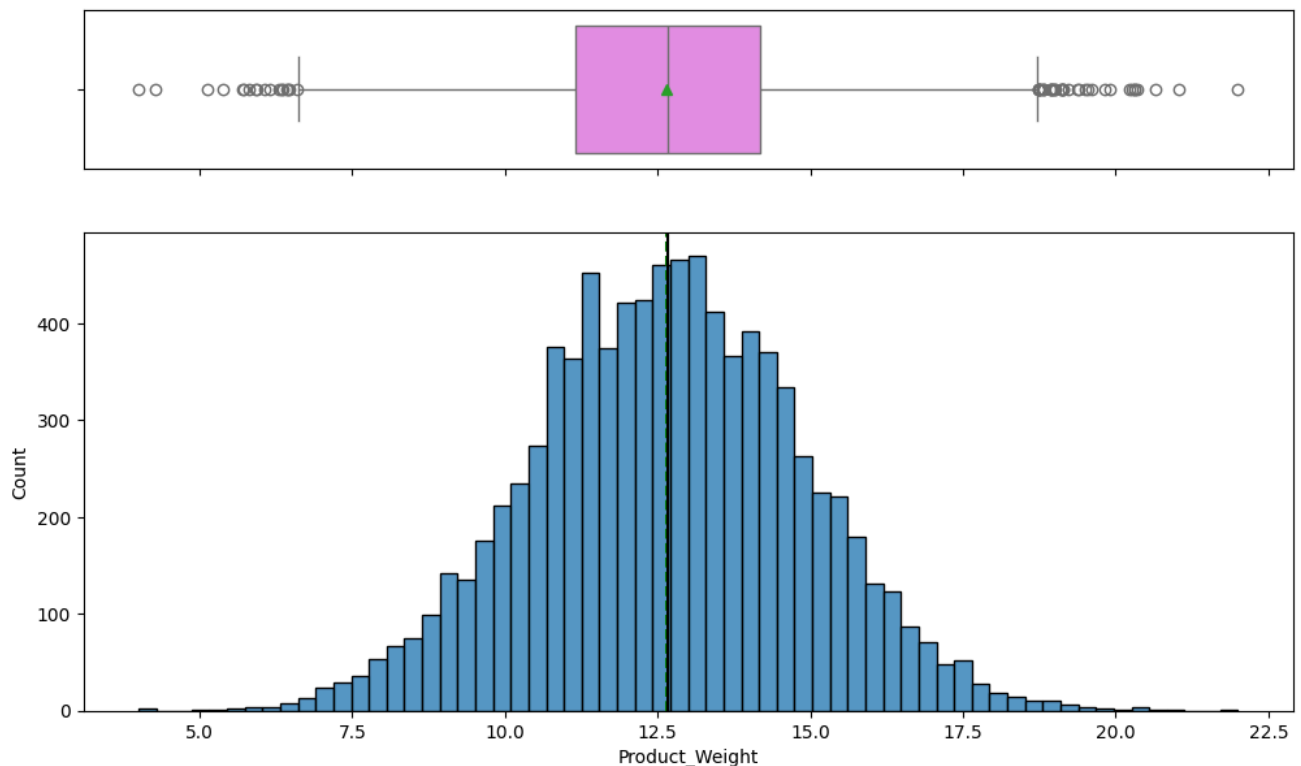
Boxplot and histogram combined

```
data: dataframe
feature: dataframe column
figsize: size of figure (default (12,7))
kde: whether to show density curve (default False)
bins: number of bins for histogram (default None)
"""

f2, (ax_box2, ax_hist2) = plt.subplots(
    nrows=2, # Number of rows of the subplot grid= 2
    sharex=True, # x-axis will be shared among all subplots
    gridspec_kw={"height_ratios": (0.25, 0.75)},
    figsize=figsize,
) # creating the 2 subplots
sns.boxplot(
    data=data, x=feature, ax=ax_box2, showmeans=True, color="violet"
) # boxplot will be created and a star will indicate the mean value of the col
sns.histplot(
    data=data, x=feature, kde=kde, ax=ax_hist2, bins=bins, palette="winter"
) if bins else sns.histplot(
    data=data, x=feature, kde=kde, ax=ax_hist2
) # For histogram
ax_hist2.axvline(
    data[feature].mean(), color="green", linestyle="--"
) # Add mean to the histogram
ax_hist2.axvline(
    data[feature].median(), color="black", linestyle="-"
) # Add median to the histogram
```

Product_Weight

```
histogram_boxplot(data, "Product_Weight")
```

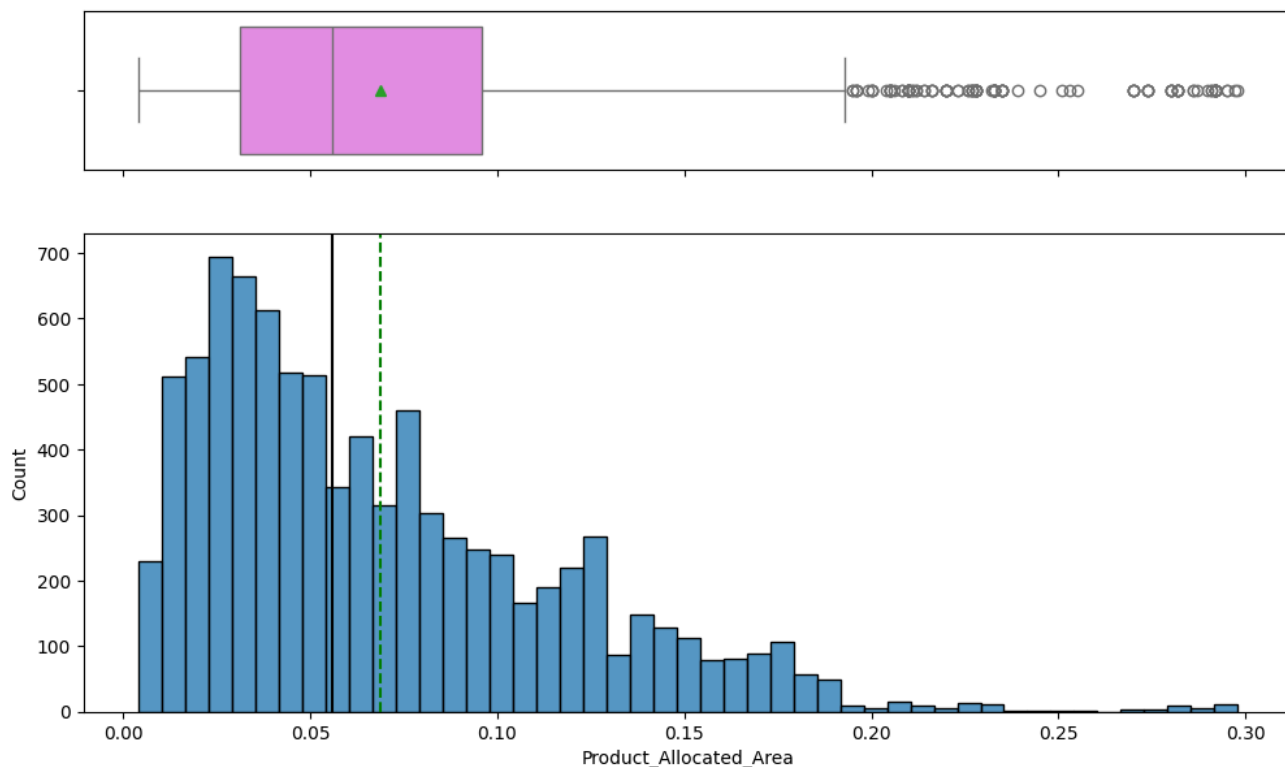


```
# Observation : The box plot and histogram shows relatively normal bell shaped dist
# The mean is ~ 12.5 suggesting the data is not skewed.
```

```
# The box plot shows there are several outliers.
```

Product_Allocated_Area

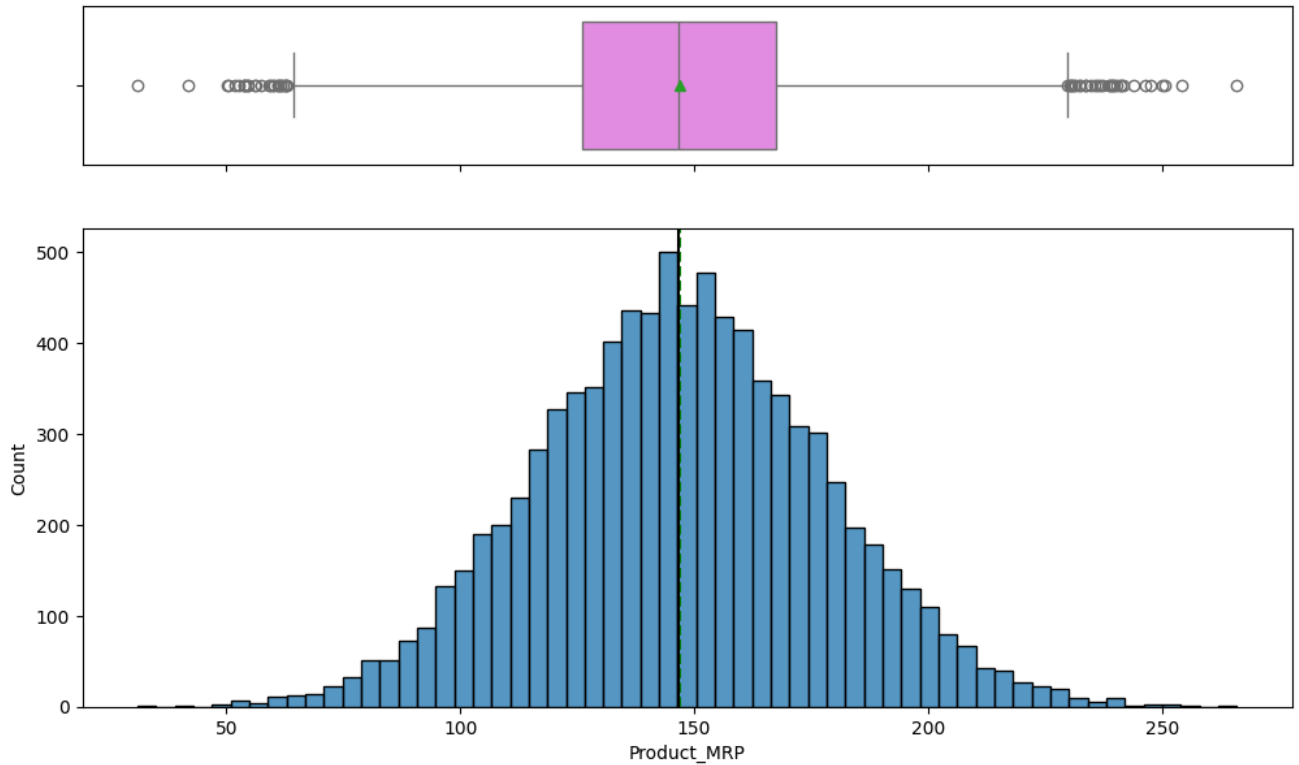
```
histogram_boxplot(data, "Product_Allocated_Area") #Complete the code to plot the box
```



```
# Observation : The histogram plot is highly right skewed meaning few products are  
# The box plot shows lot of outliers on the higher end which also represents Superk
```

Product_MRP

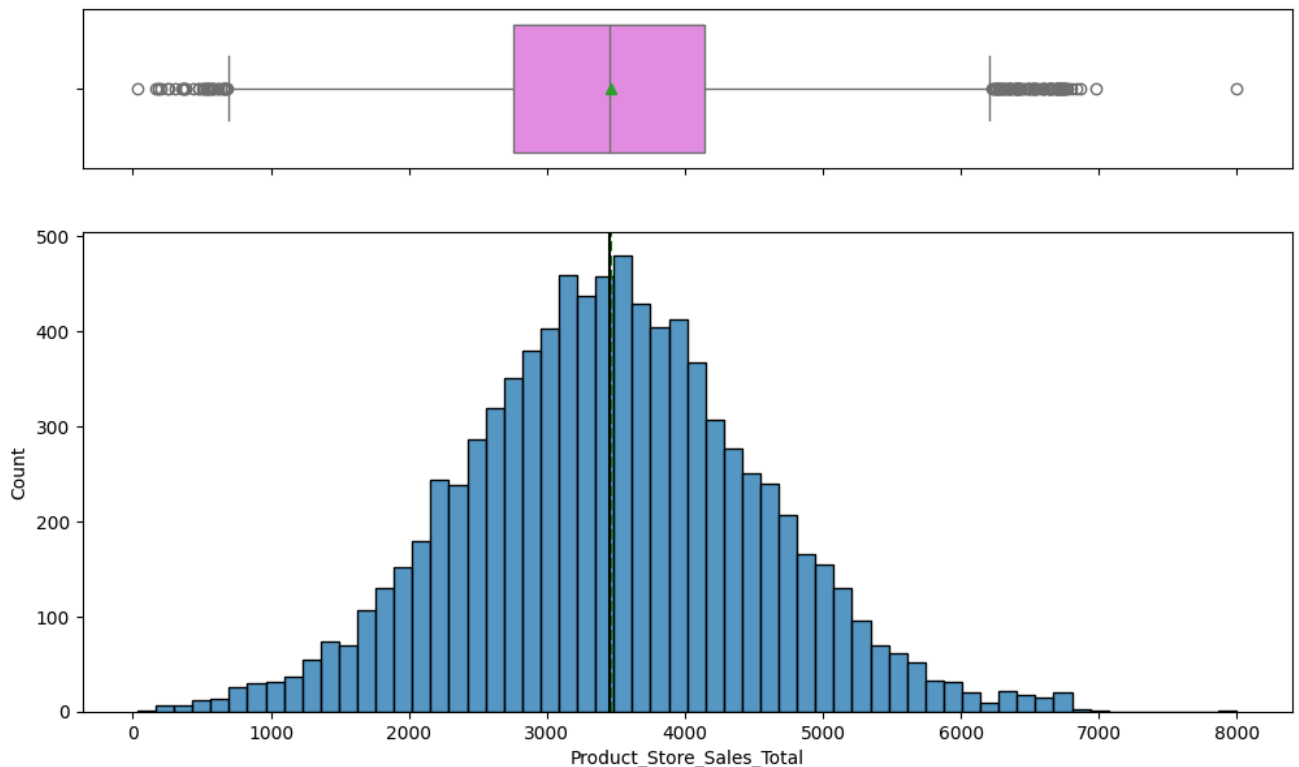
```
histogram_boxplot(data, "Product_MRP")
```

Observation : The Product_MRP histogram shows a multimodal distribution with several peaks.
The boxplot for Product_MRP shows no outliers.

Product_Store_Sales_Total

`histogram_boxplot(data, "Product_Store_Sales_Total")` *#Complete the code to plot the histogram and boxplot.*



#Observation : The histogram shows a relatively symmetric, bell-shaped curve, indicating a normal distribution.
The boxplot highlights several outliers on both the extreme low and extreme high ends.
The bulk of the sales fall approximately between 2,700 and 4,200.

```

# function to create labeled barplots

def labeled_barplot(data, feature, perc=False, n=None):
    """
    Barplot with percentage at the top

    data: dataframe
    feature: dataframe column
    perc: whether to display percentages instead of count (default is False)
    n: displays the top n category levels (default is None, i.e., display all levels)
    """

    total = len(data[feature]) # length of the column
    count = data[feature].nunique()
    if n is None:
        plt.figure(figsize=(count + 1, 5))
    else:
        plt.figure(figsize=(n + 1, 5))

    plt.xticks(rotation=90, fontsize=15)
    ax = sns.countplot(
        data=data,
        x=feature,
        palette="Paired",
        order=data[feature].value_counts().index[:n].sort_values(),
    )

    for p in ax.patches:
        if perc == True:
            label = "{:.1f}%".format(
                100 * p.get_height() / total
            ) # percentage of each class of the category
        else:
            label = p.get_height() # count of each level of the category

        x = p.get_x() + p.get_width() / 2 # width of the plot
        y = p.get_height() # height of the plot

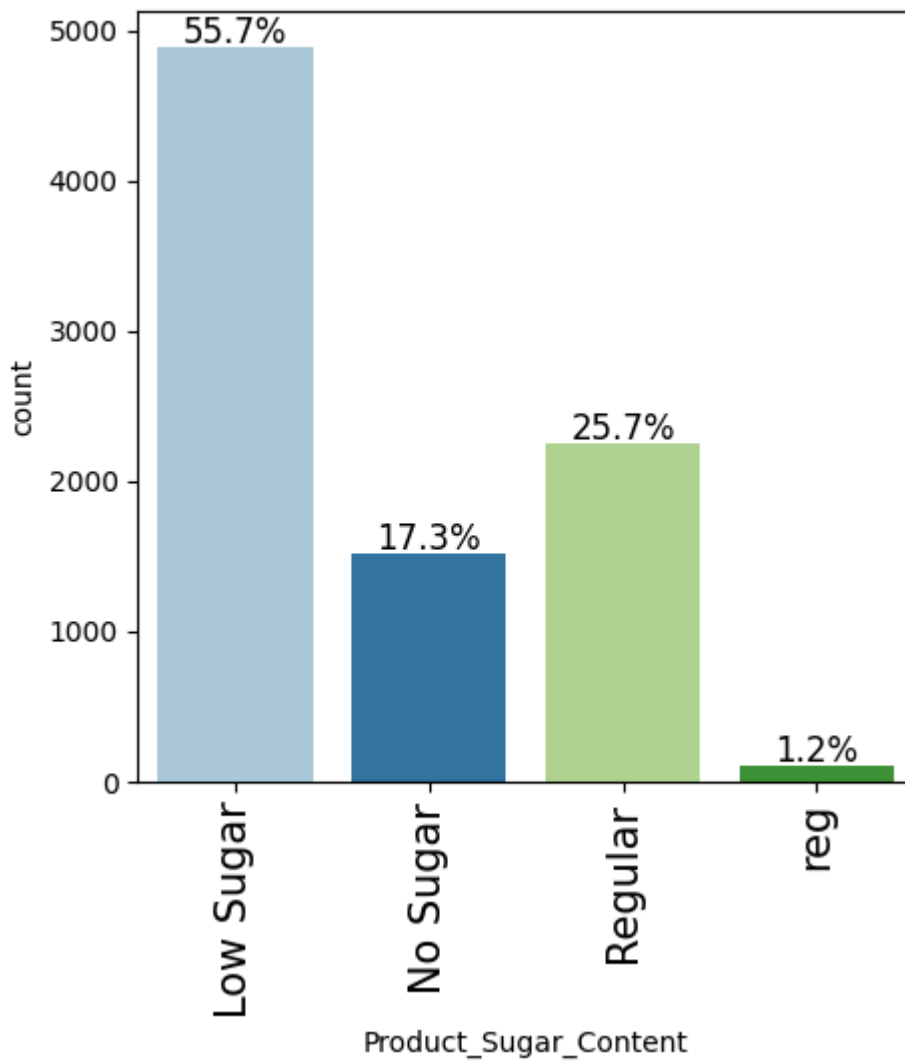
        ax.annotate(
            label,
            (x, y),
            ha="center",
            va="center",
            size=12,
            xytext=(0, 5),
            textcoords="offset points",
        ) # annotate the percentage

    plt.show() # show the plot

```

Product_Sugar_Content

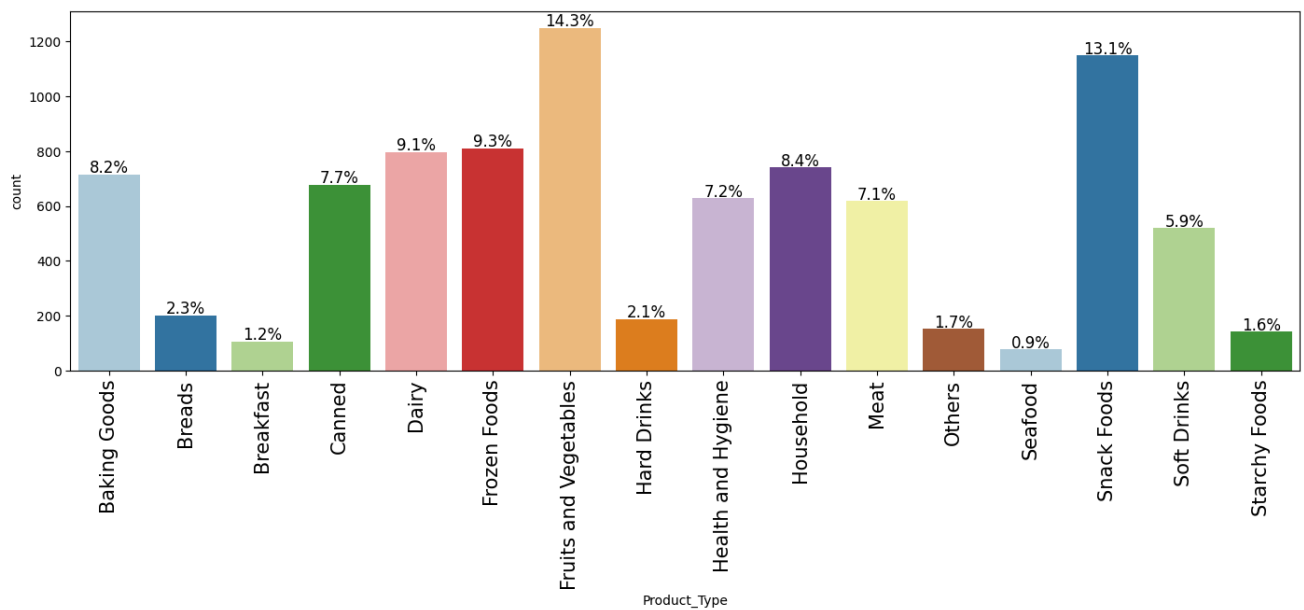
```
labeled_barplot(data, "Product_Sugar_Content", perc=True)
```



```
# Observation : More than 50% items falls under the low Sugar content.  
# Only 17.3 % are no sugar.  
# The Regular and reg needs to be merged , should be the same.
```

Product_Type

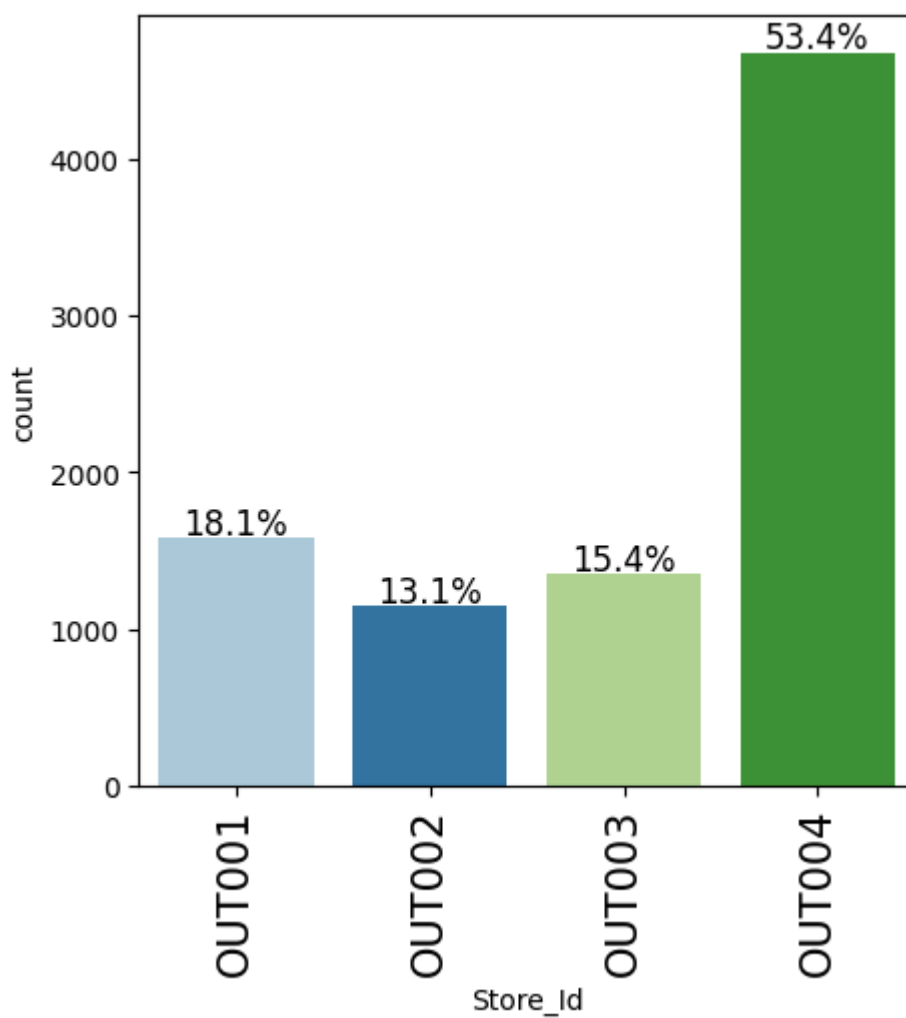
```
labeled_barplot(data, "Product_Type", perc=True)
```



#Observation : The top product type is Fruits and Vegetables at 14.3% followed with
Interesting to observe Dairy and Frozen Foods are almost at the same percentage .

Store_Id

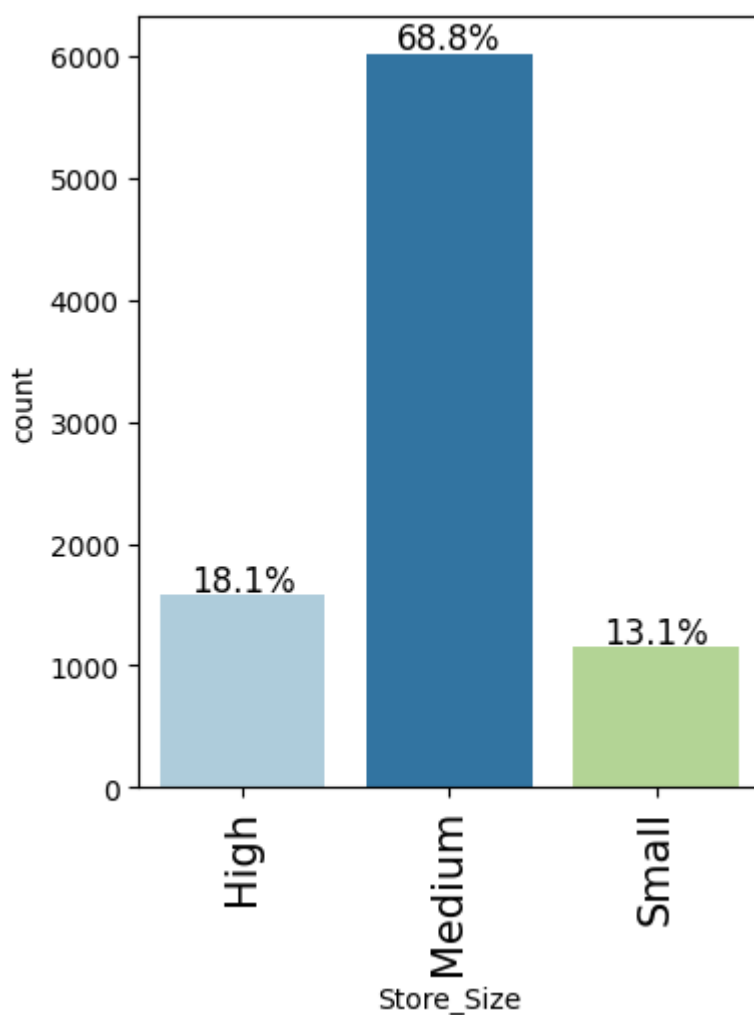
```
labeled_barplot(data, "Store_Id", perc=True)
```



Observation : The store OUT004 is most frequent identifier representing majority
All other stores are almost the same.
This can mean data imbalance. The heavy concentration in one store suggests the n

Store_Size

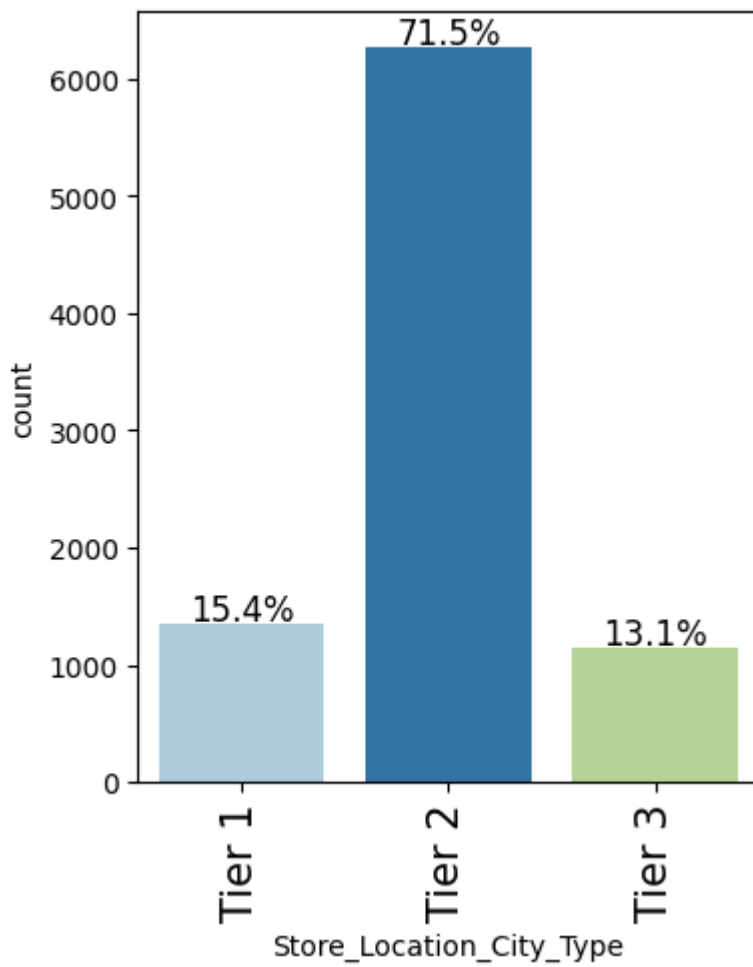
```
labeled_barplot(data, "Store_Size", perc=True)
```



Observation : The medium sized store has majority of enteries at 68.8%.
Even here the model prediction can heavily depend on the Medium sized store patte

Store_Location_City_Type

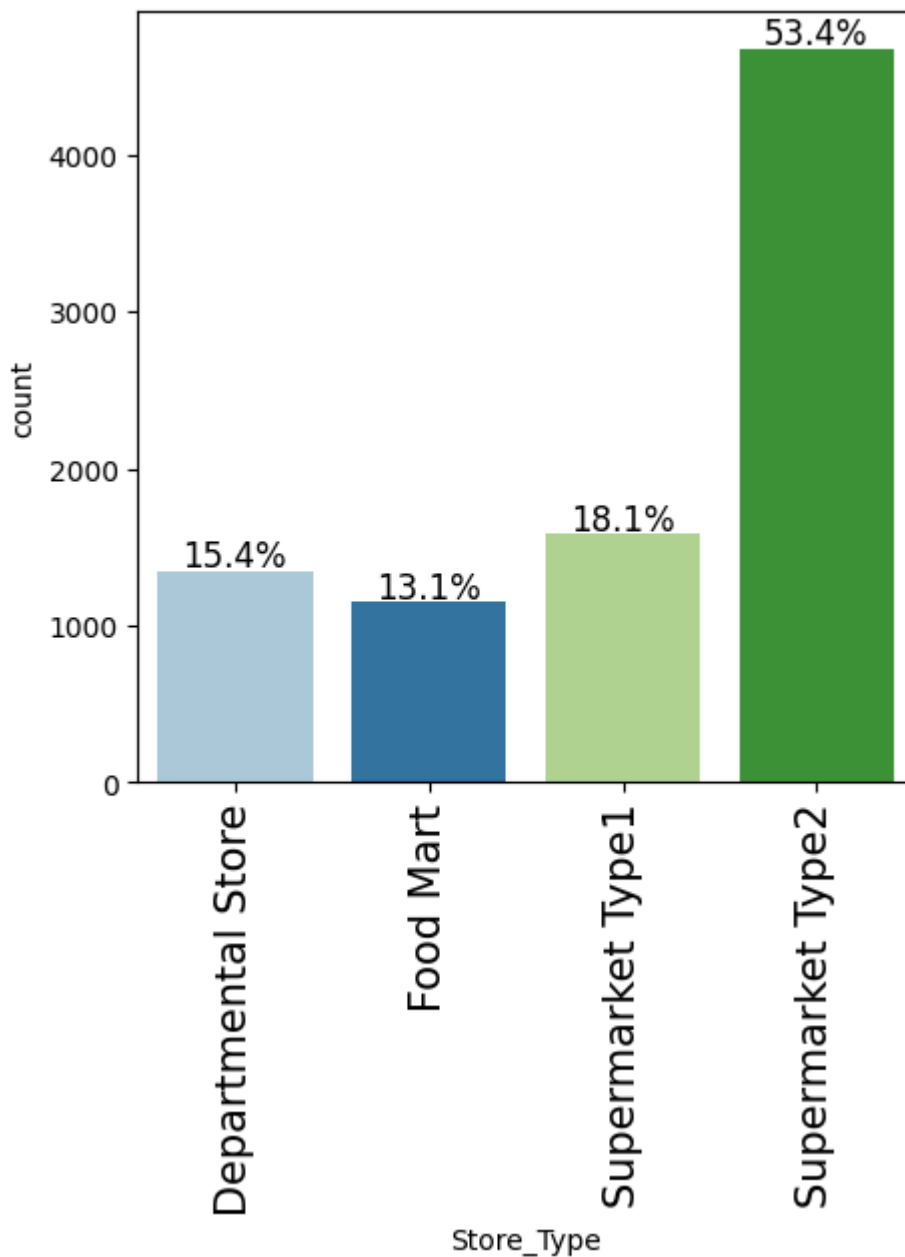
```
labeled_barplot(data, "Store_Location_City_Type", perc=True)
```



*# Observation : Tier 2 cities represent the overwhelming majority of the dataset at
There can be a data imbalance due to this.*

Store_Type

```
labeled_barplot(data, "Store_Type", perc=True)
```



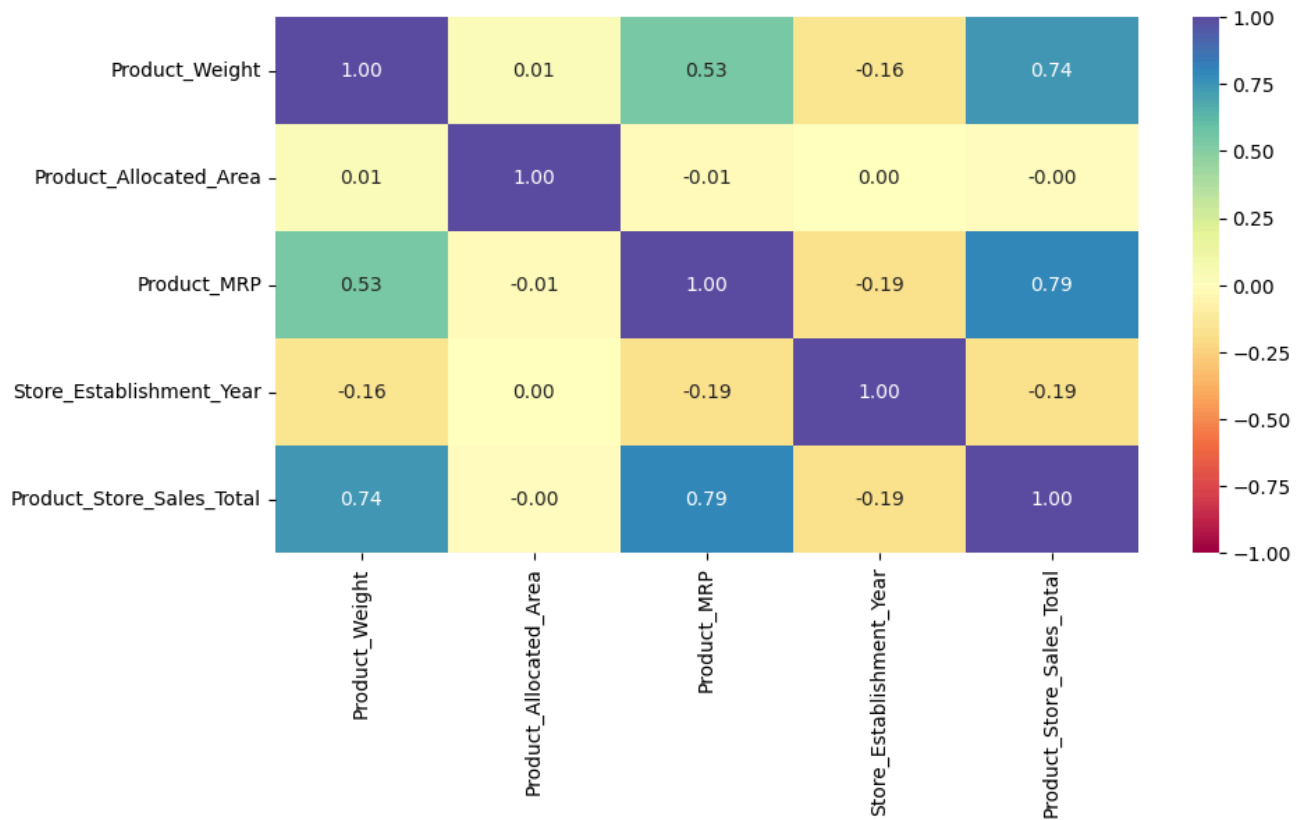
*# Observation :The supermarket Type 2 represent the majority of the dataset at 53.
Need to find the difference between Supermarket Type 1 and Type 2 to predict the*

Bivariate Analysis

Correlation matrix

```
cols_list = data.select_dtypes(include=np.number).columns.tolist()

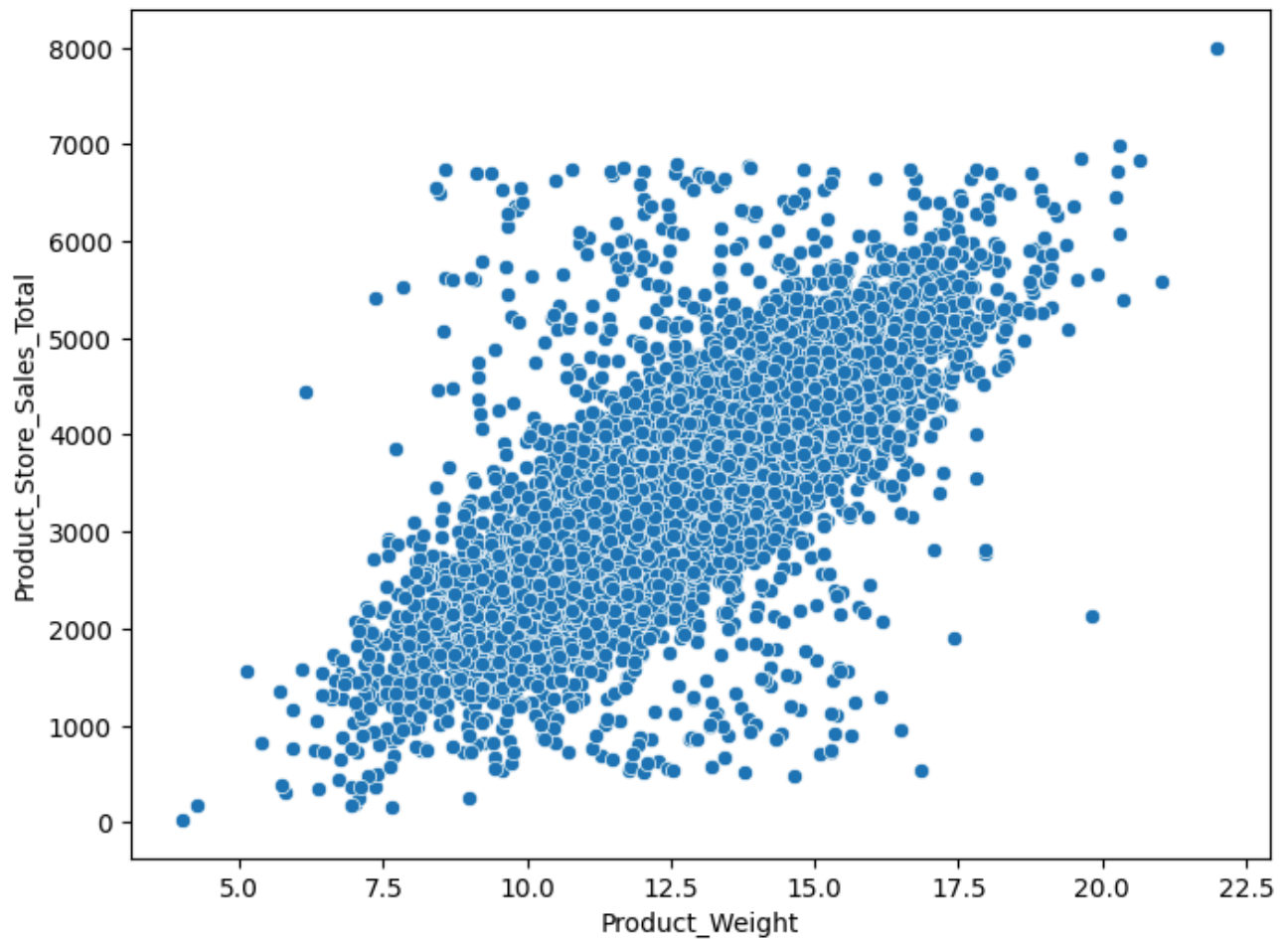
plt.figure(figsize=(10, 5))
sns.heatmap(
    data[cols_list].corr(), annot=True, vmin=-1, vmax=1, fmt=".2f", cmap="Spectral"
)
plt.show()
```



```
# Observation
# Product_MRP and Product_Weight show strong positive correlations (0.79 and 0.74 r
# Product_Allocated_Area has a near-zero correlation with sales, suggesting that th
# Store_Establishment_Year displays weak negative correlations across all variables
```

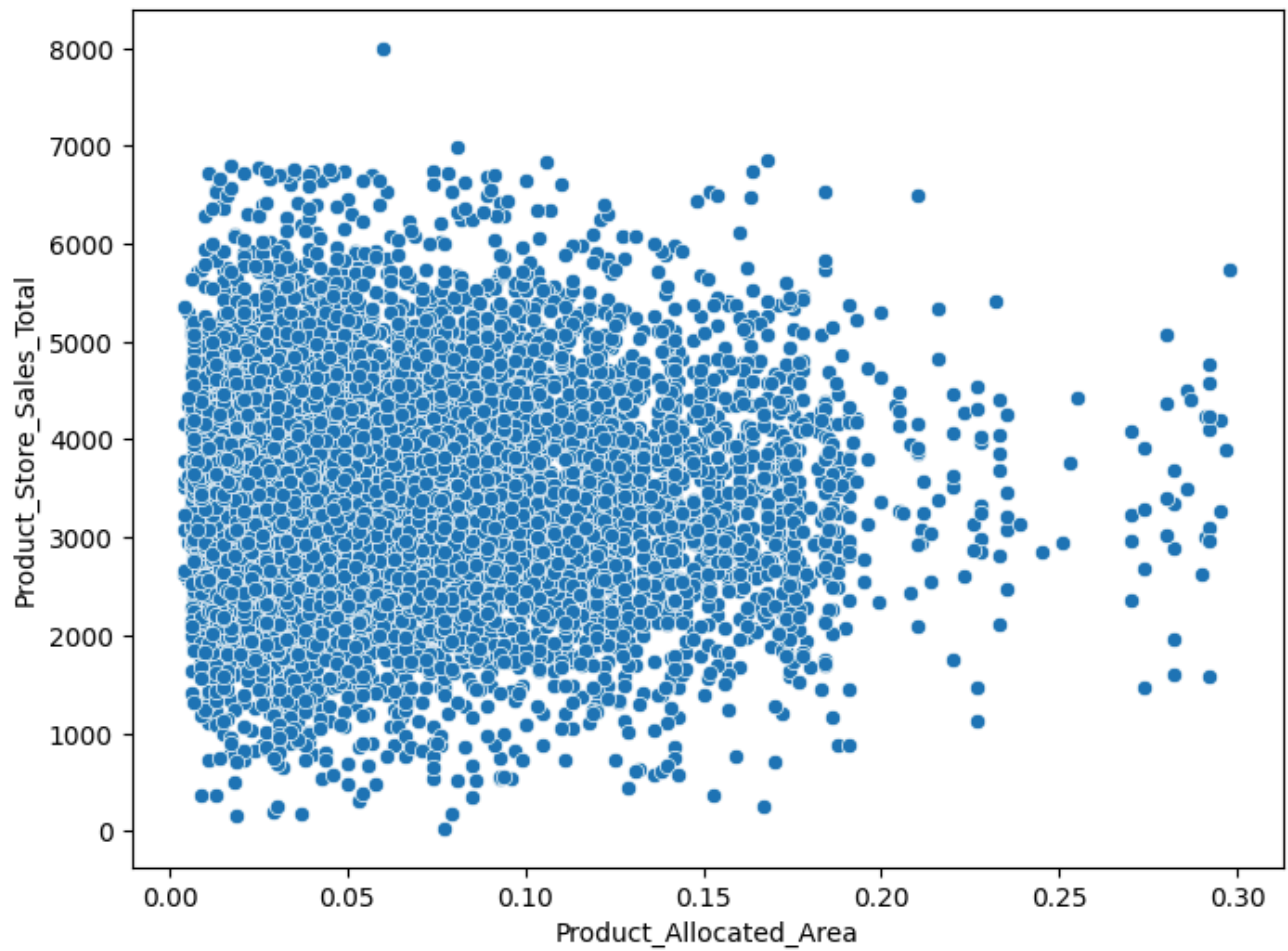
**Let's check the distribution of our target variable i.e
Product_Store_Sales_Total with the numeric columns**

```
plt.figure(figsize=[8, 6])
sns.scatterplot(x=data.Product_Weight, y=data.Product_Store_Sales_Total)
plt.show()
```

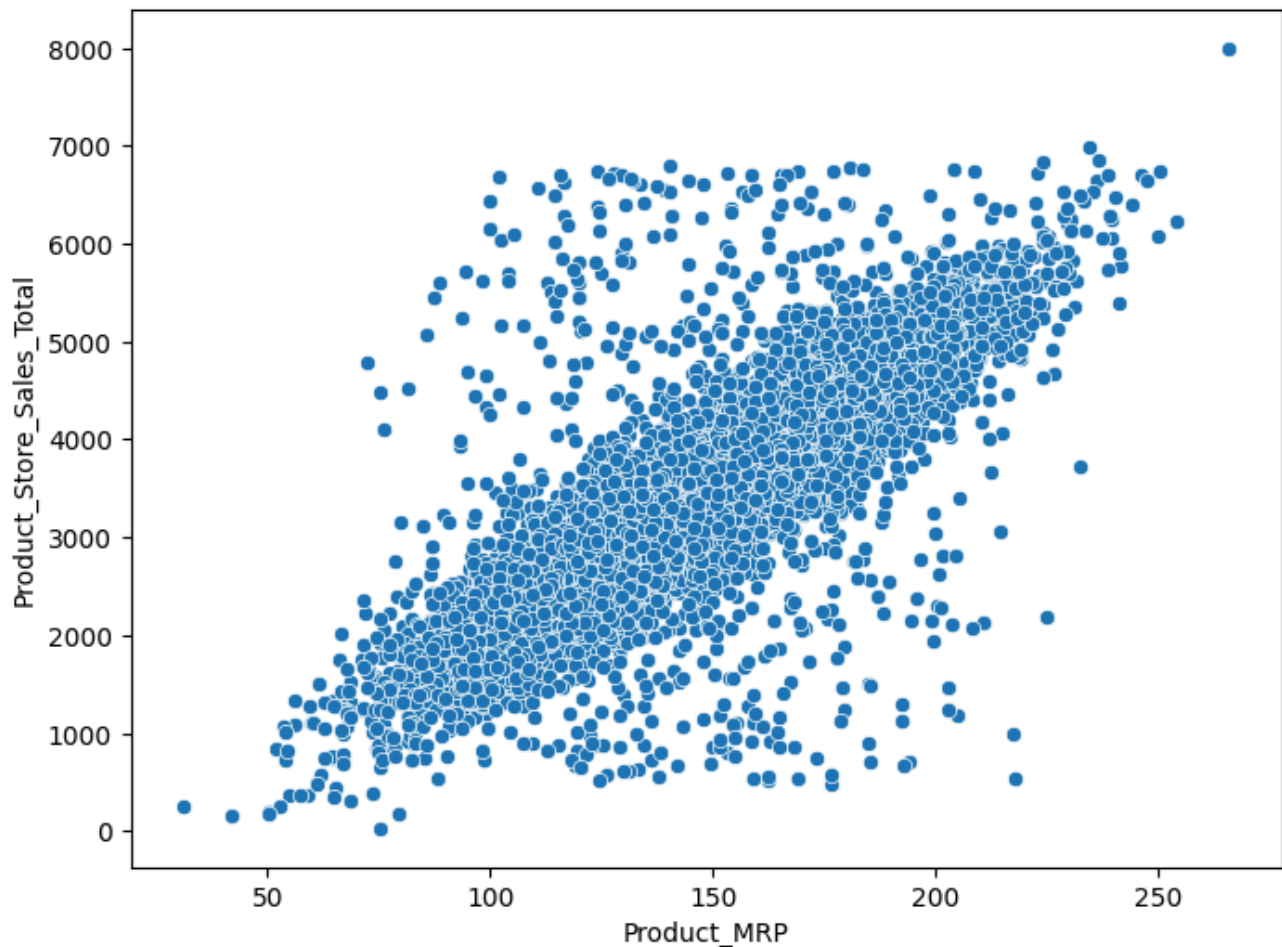
*# Observation: A change in a product's weight consistently corresponds to a change
A high concentration of products generating revenue between \$2,000\$ and \$6,000\$,
The spread of sales remains relatively consistent regardless of product weight, s*

```
plt.figure(figsize=[8, 6])  
sns.scatterplot(x=data.Product_Allocated_Area, y=data.Product_Store_Sales_Total)  
plt.show()
```



Observation : The plot shows a vertical dense cluster with no visible slope, conf
The distribution suggests that product visibility within the store is less import

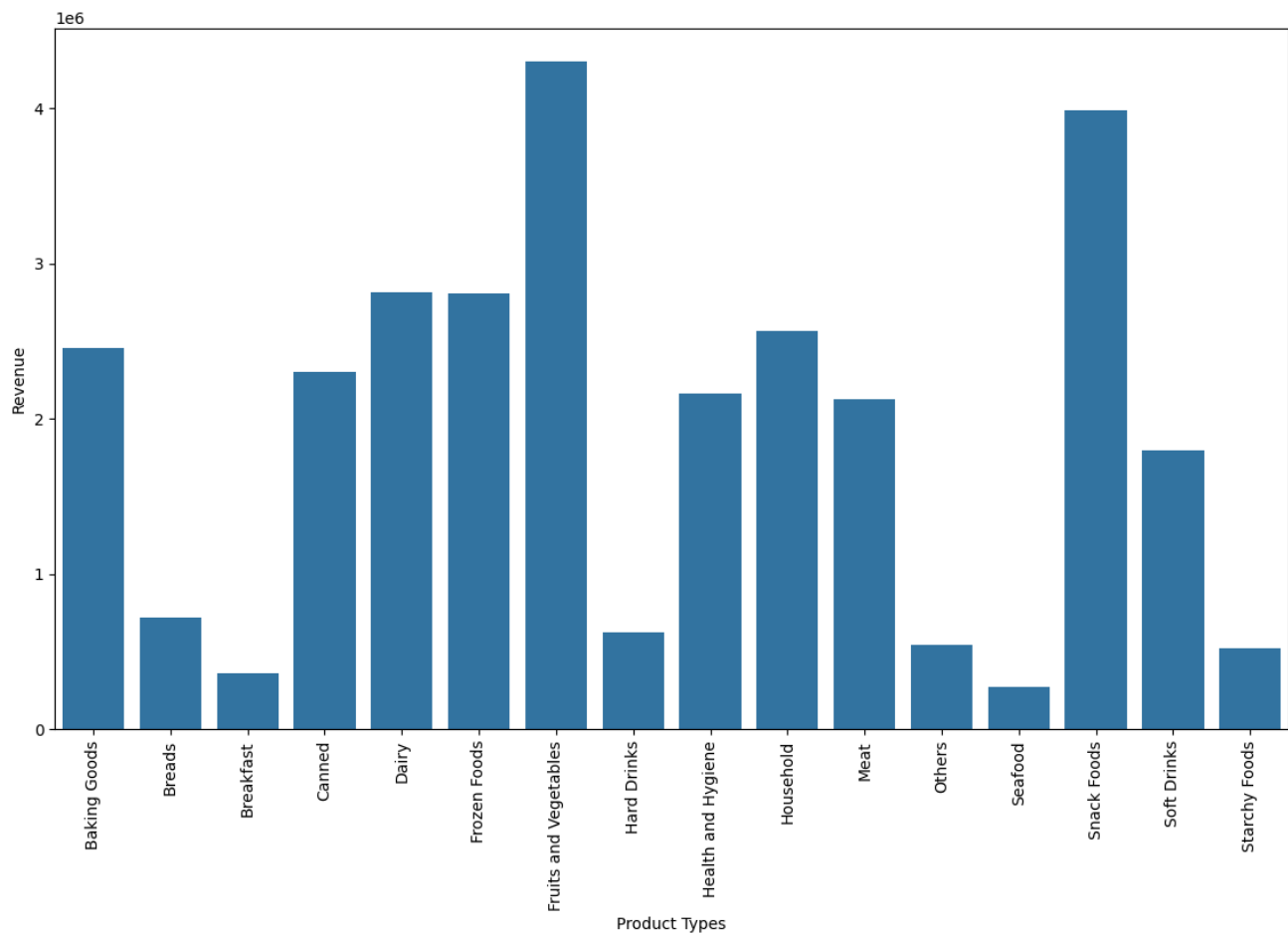
```
plt.figure(figsize=[8, 6])  
sns.scatterplot(x=data.Product_MRP, y=data.Product_Store_Sales_Total)  
plt.show()
```



*# Observation : The plot shows a clear upward slope, confirming that as Product_MRP
The data is grouped into four visible vertical clusters, reflecting the multimodal*

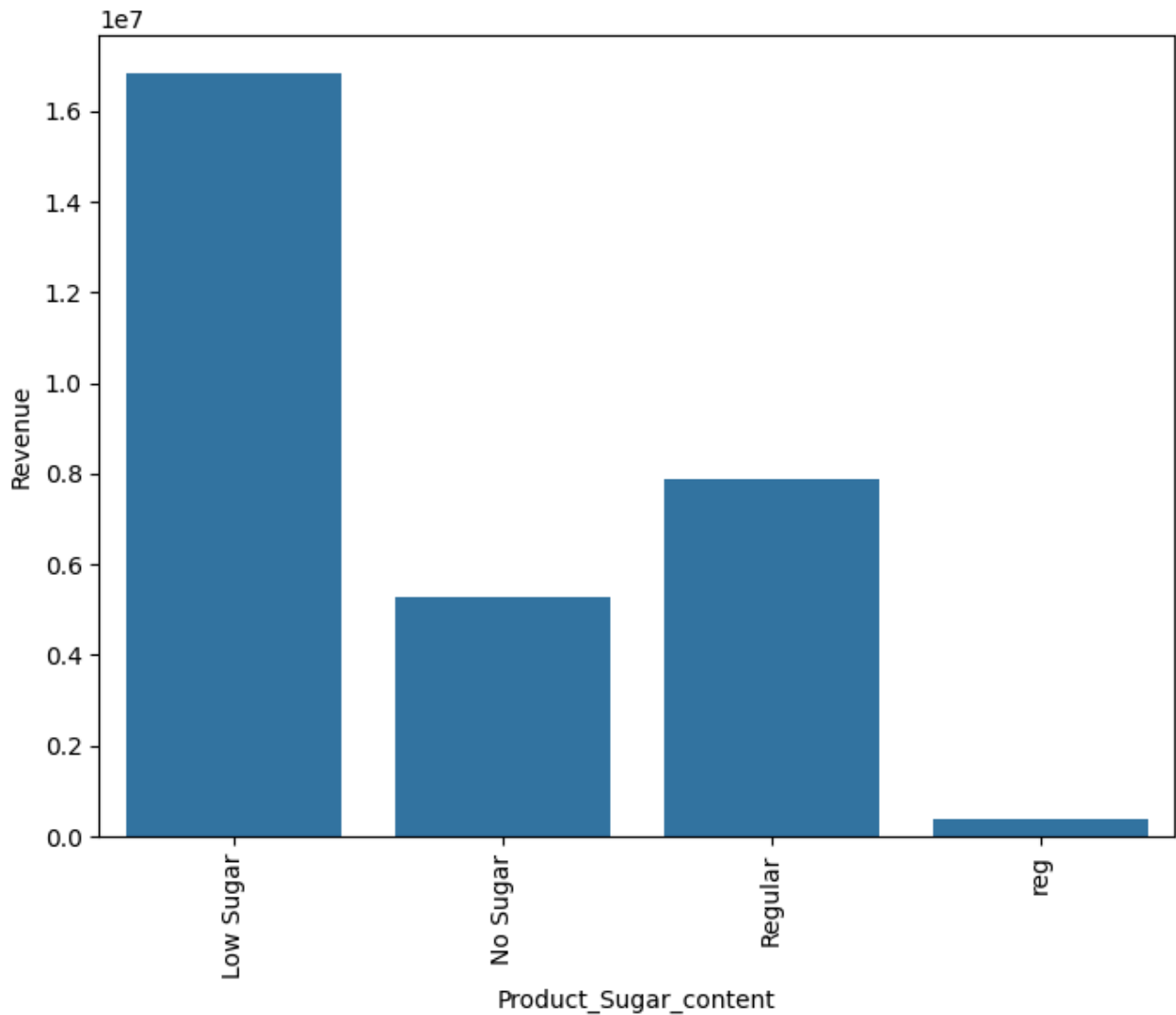
Let us see from which product type the company is generating most of the revenue

```
df_revenue1 = data.groupby(["Product_Type"], as_index=False)[
    "Product_Store_Sales_Total"
].sum()
plt.figure(figsize=[14, 8])
plt.xticks(rotation=90)
a = sns.barplot(x=df_revenue1.Product_Type, y=df_revenue1.Product_Store_Sales_Total)
a.set_xlabel("Product Types")
a.set_ylabel("Revenue")
plt.show()
```



Observation : Fruits and Vegetables and Snack foods generate the highest revenue

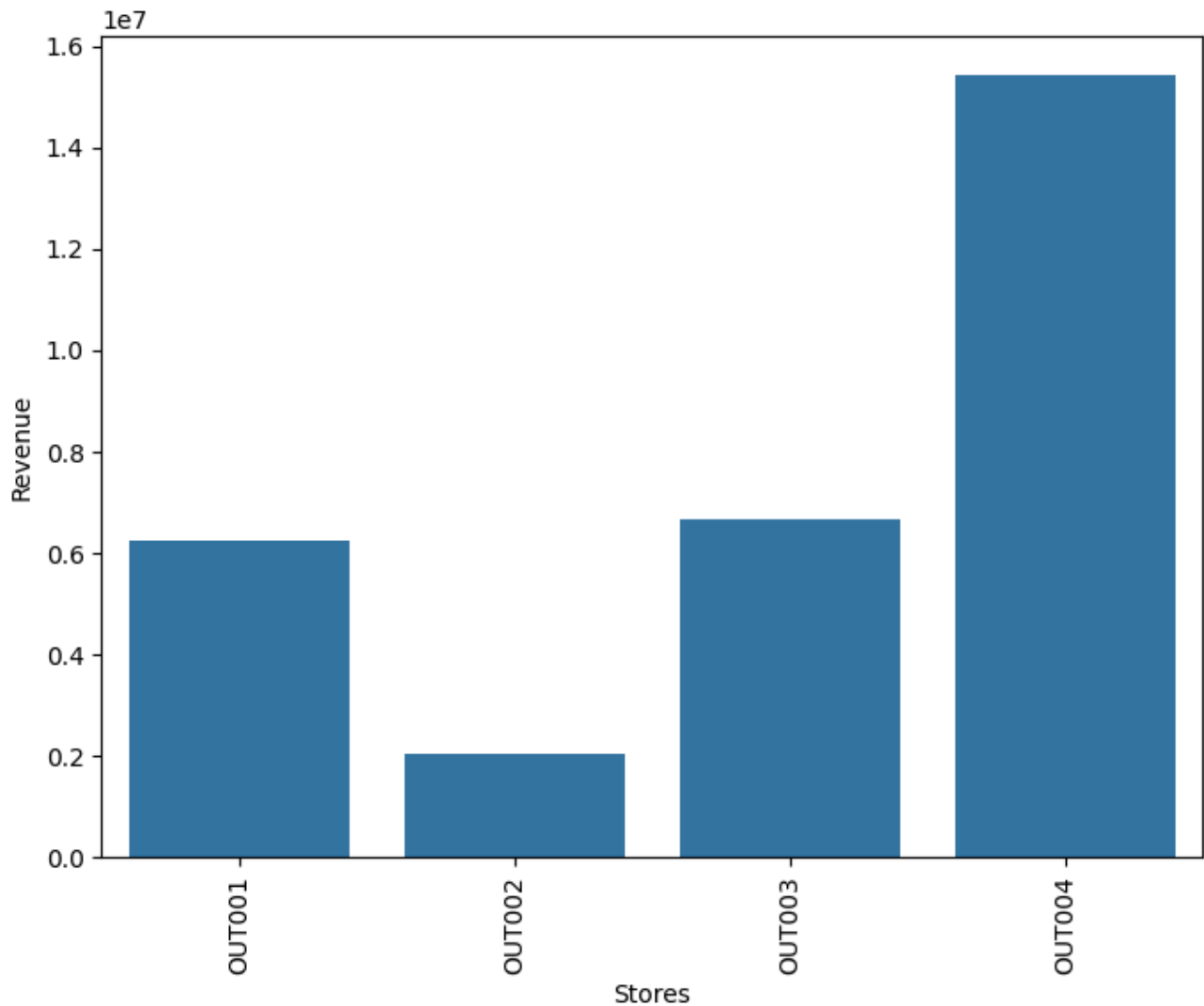
```
df_revenue2 = data.groupby(["Product_Sugar_Content"], as_index=False)[
    "Product_Store_Sales_Total"
].sum()
plt.figure(figsize=[8, 6])
plt.xticks(rotation=90)
b = sns.barplot(
    x=df_revenue2.Product_Sugar_Content, y=df_revenue2.Product_Store_Sales_Total
)
b.set_xlabel("Product_Sugar_content")
b.set_ylabel("Revenue")
plt.show()
```



Observation : the Food with low sugar generates the highest revenue.

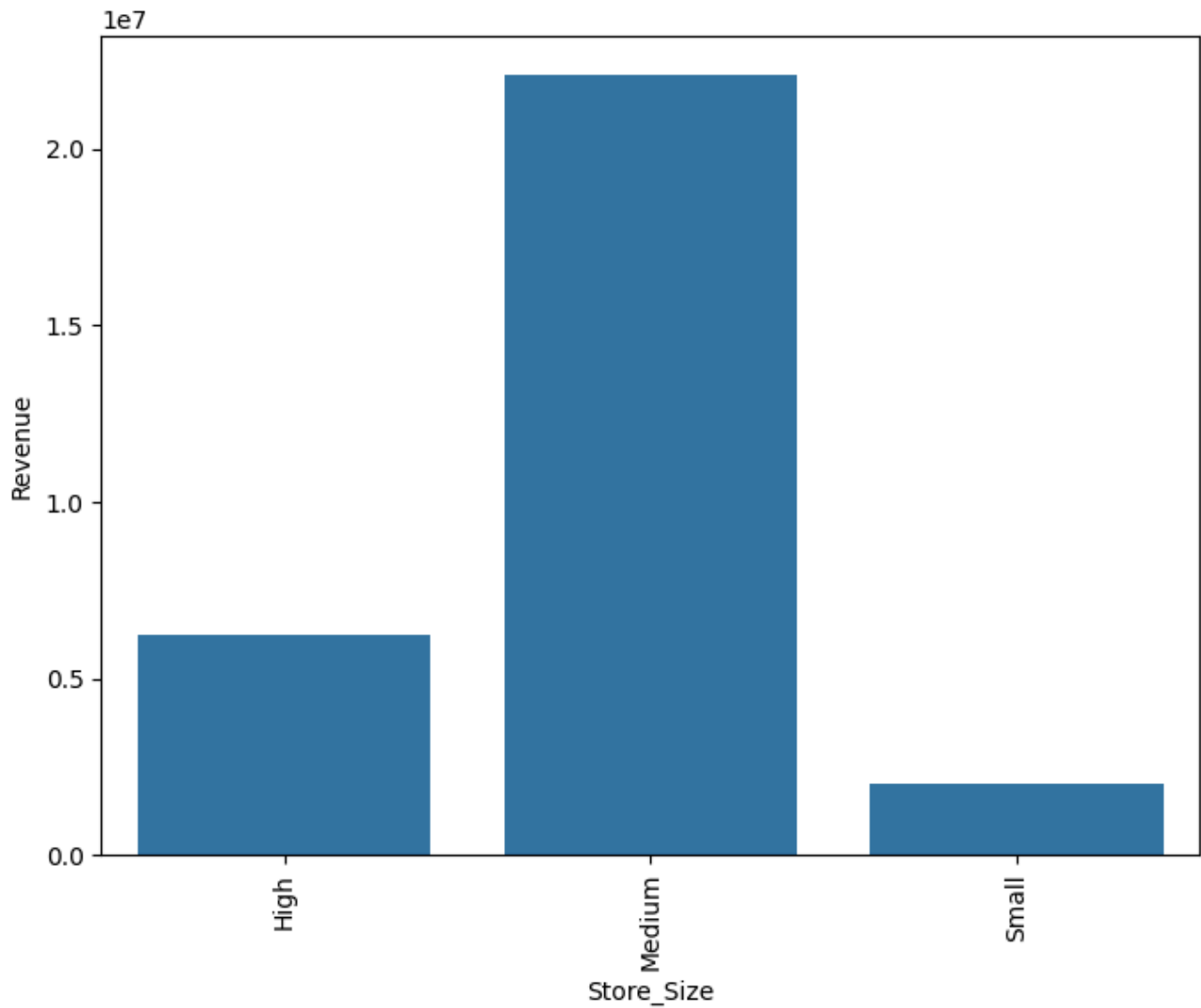
Let us see from which type of stores and locations the revenue generation is more.

```
df_store_revenue = data.groupby(["Store_Id"], as_index=False)[
    "Product_Store_Sales_Total"
].sum()
plt.figure(figsize=[8, 6])
plt.xticks(rotation=90)
r = sns.barplot(
    x=df_store_revenue.Store_Id, y=df_store_revenue.Product_Store_Sales_Total
)
r.set_xlabel("Stores")
r.set_ylabel("Revenue")
plt.show()
```



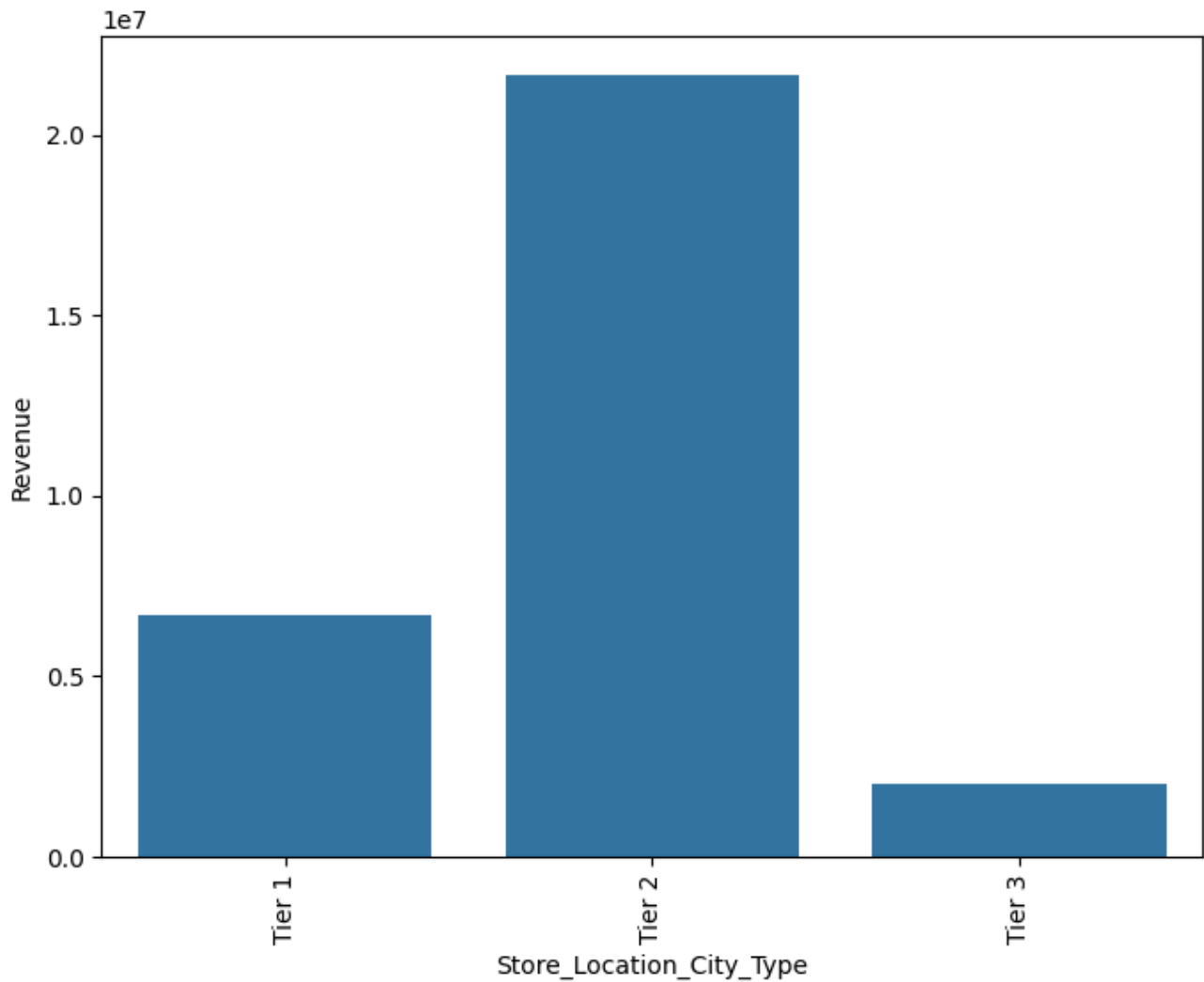
Observation : The Store id OUT004 has the highest revenue generation.

```
df_revenue3 = data.groupby(["Store_Size"], as_index=False)[
    "Product_Store_Sales_Total"
].sum()
plt.figure(figsize=[8, 6])
plt.xticks(rotation=90)
c = sns.barplot(x=df_revenue3.Store_Size, y=df_revenue3.Product_Store_Sales_Total)
c.set_xlabel("Store_Size")
c.set_ylabel("Revenue")
plt.show()
```



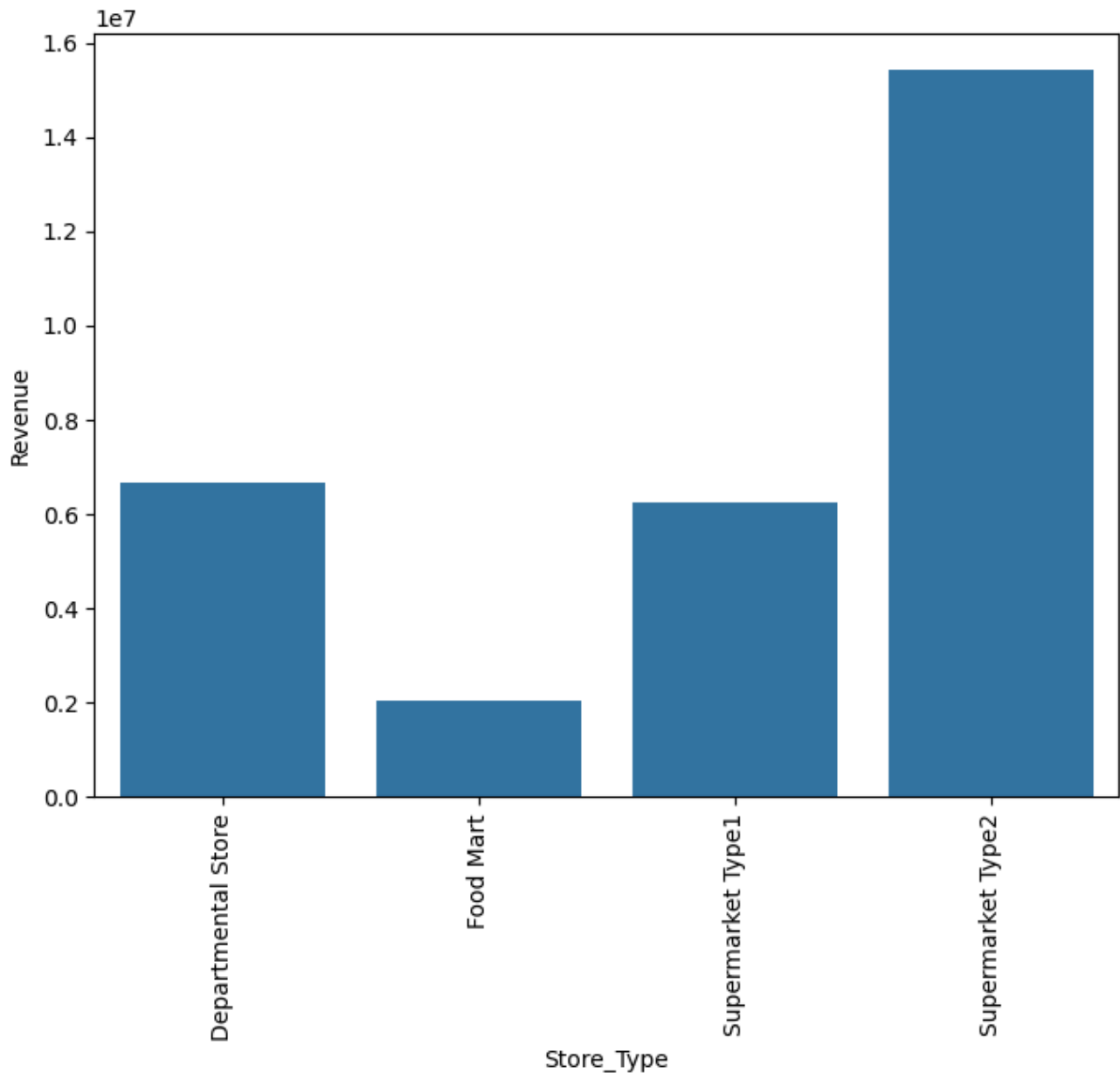
Observation : The Store size meduim has the highest revenue generation.

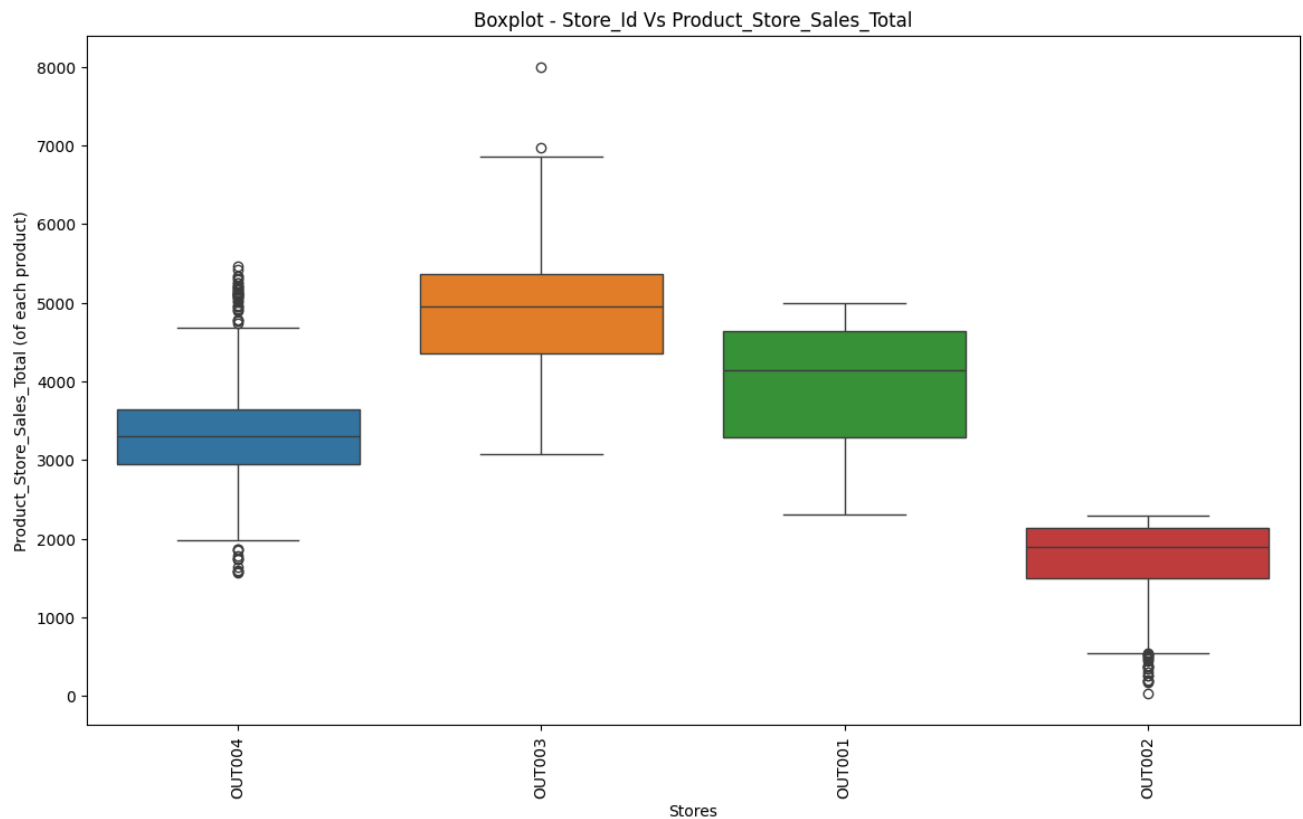
```
df_revenue4 = data.groupby(["Store_Location_City_Type"], as_index=False)[
    "Product_Store_Sales_Total"
].sum()
plt.figure(figsize=[8, 6])
plt.xticks(rotation=90)
d = sns.barplot(
    x=df_revenue4.Store_Location_City_Type, y=df_revenue4.Product_Store_Sales_Total
)
d.set_xlabel("Store_Location_City_Type")
d.set_ylabel("Revenue")
plt.show()
```

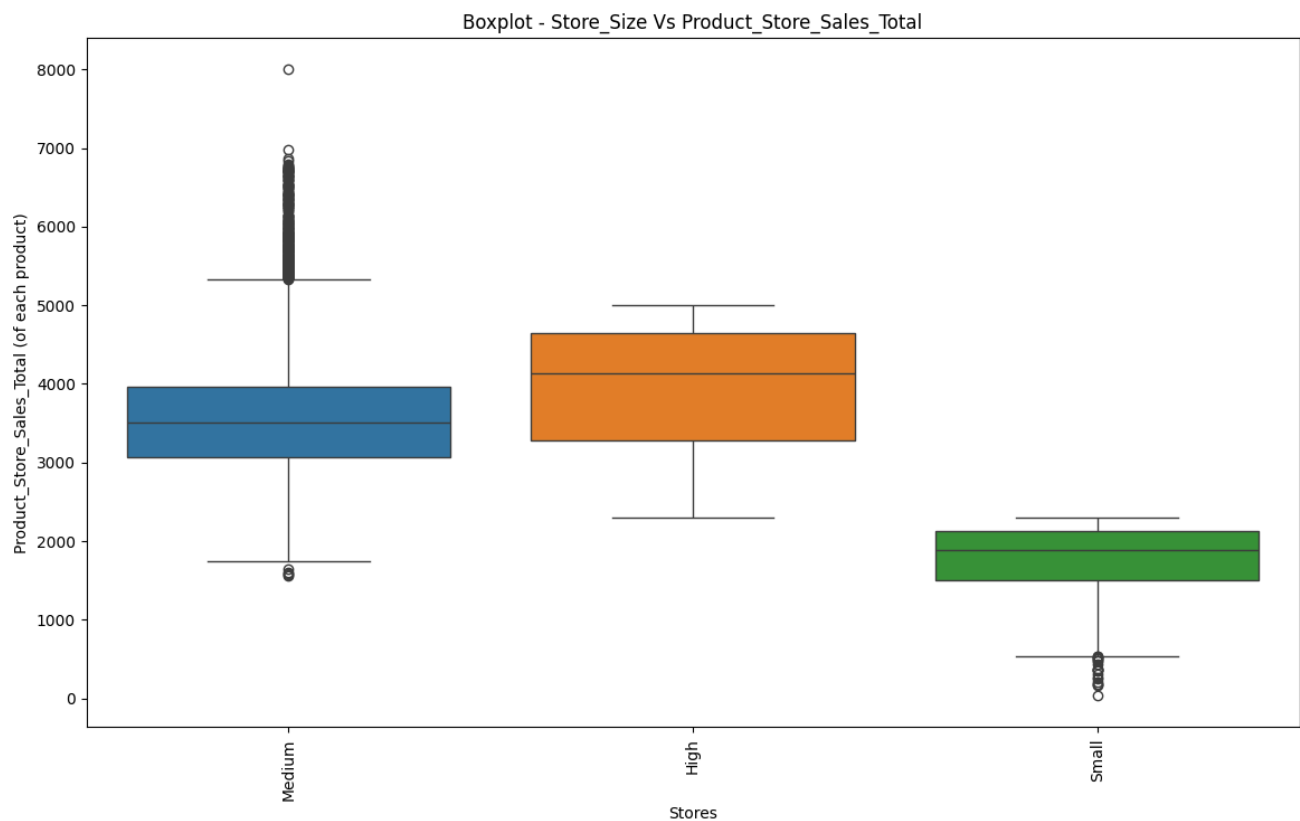


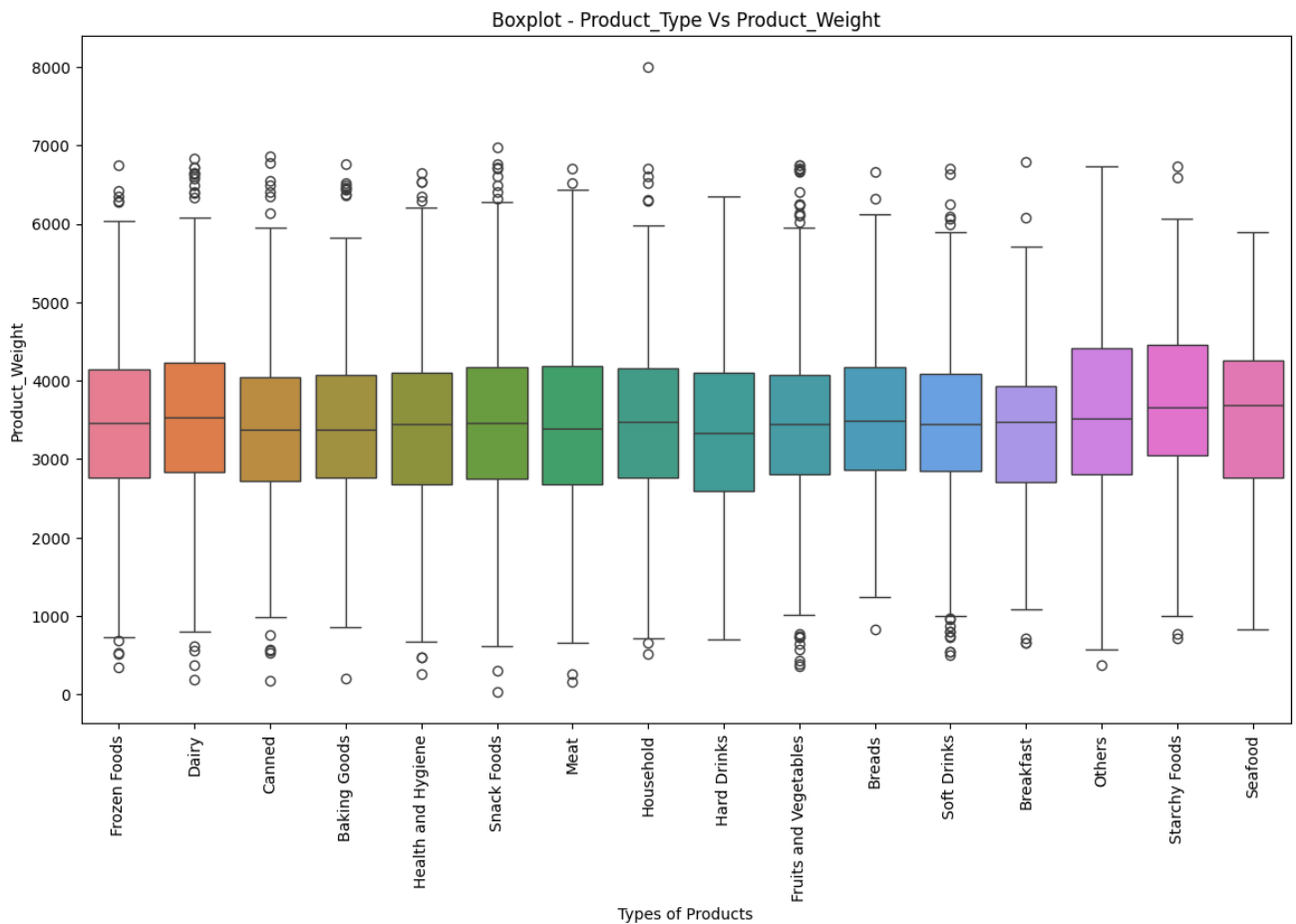
Observation : The Store location city of type Tier 2 has the highest revenue generated

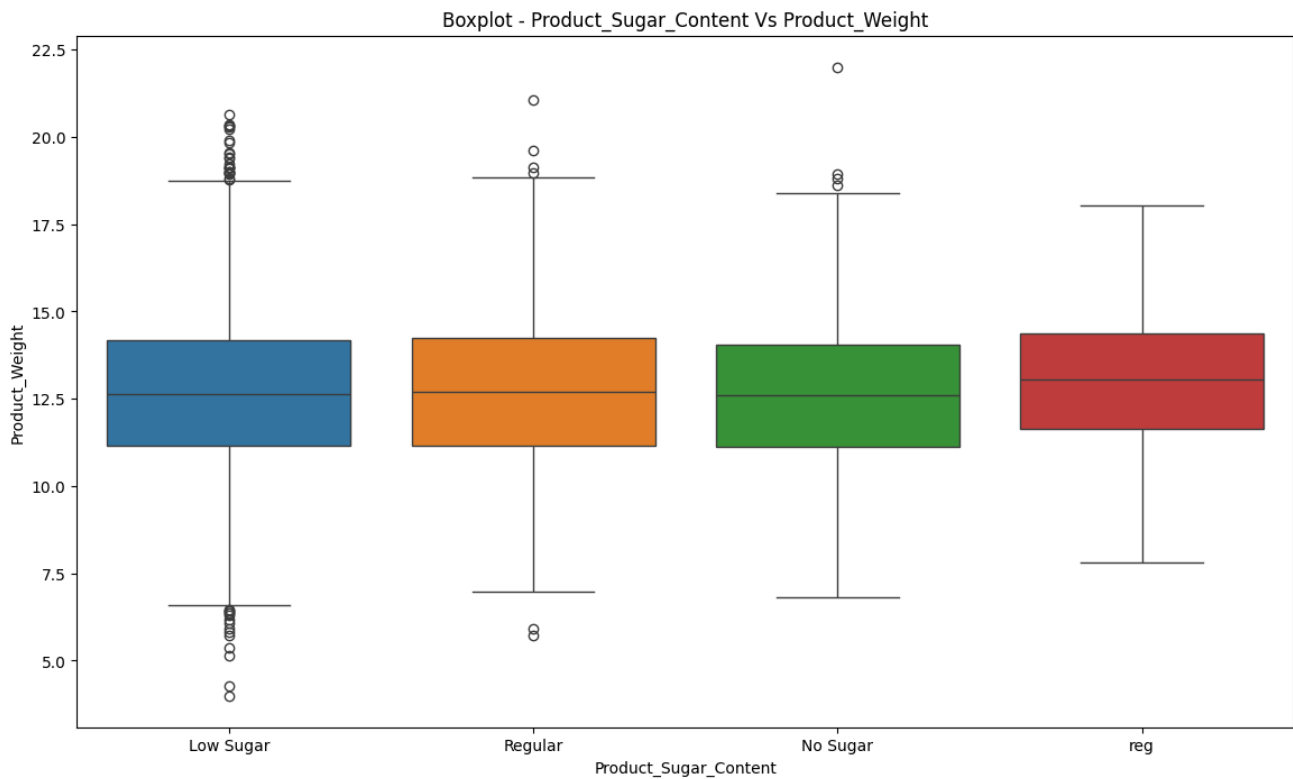
```
df_revenue5 = data.groupby(["Store_Type"], as_index=False)[
    "Product_Store_Sales_Total"
].sum()
plt.figure(figsize=[8, 6])
plt.xticks(rotation=90)
e = sns.barplot(x=df_revenue5.Store_Type, y=df_revenue5.Product_Store_Sales_Total)
e.set_xlabel("Store_Type")
e.set_ylabel("Revenue")
plt.show()
```

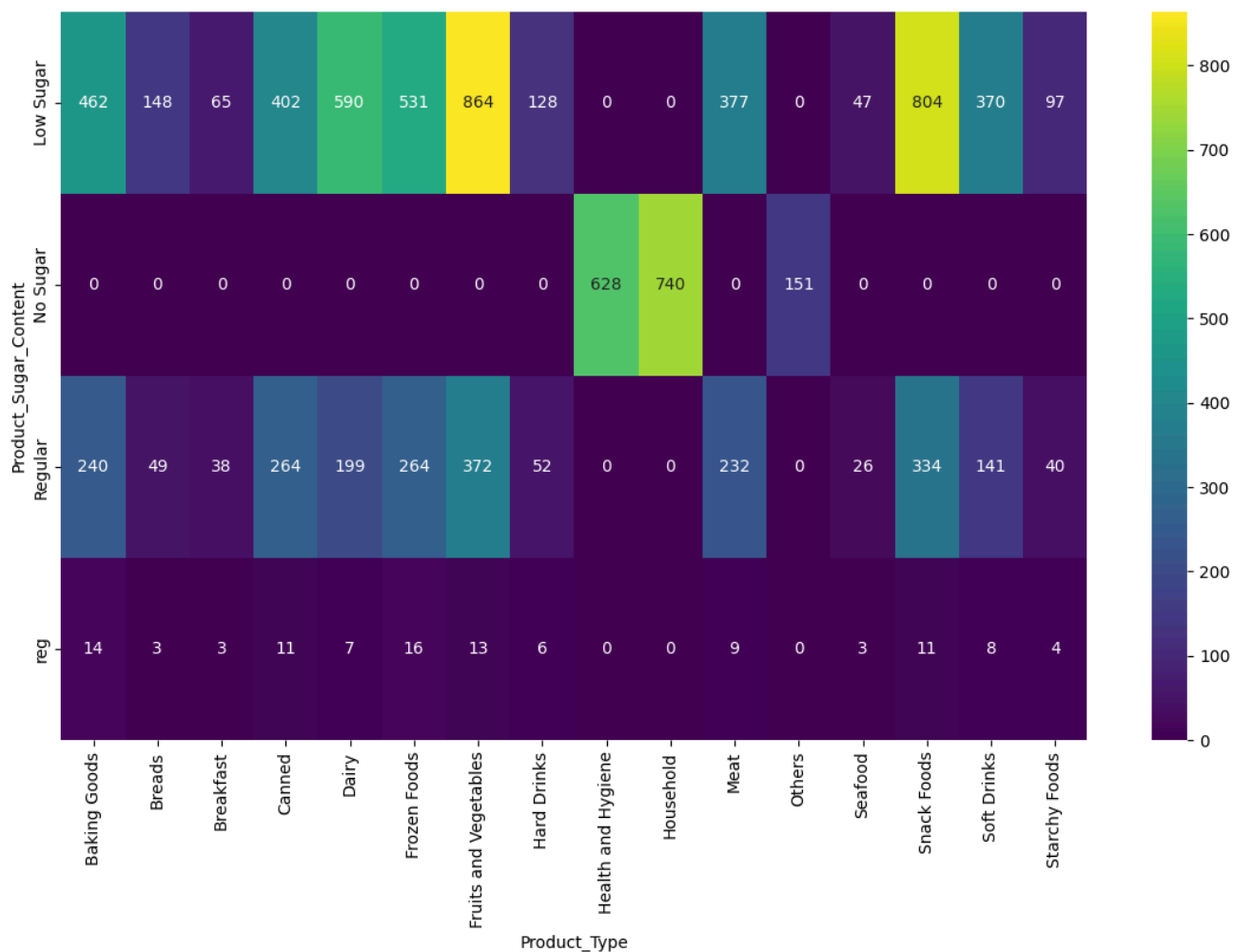



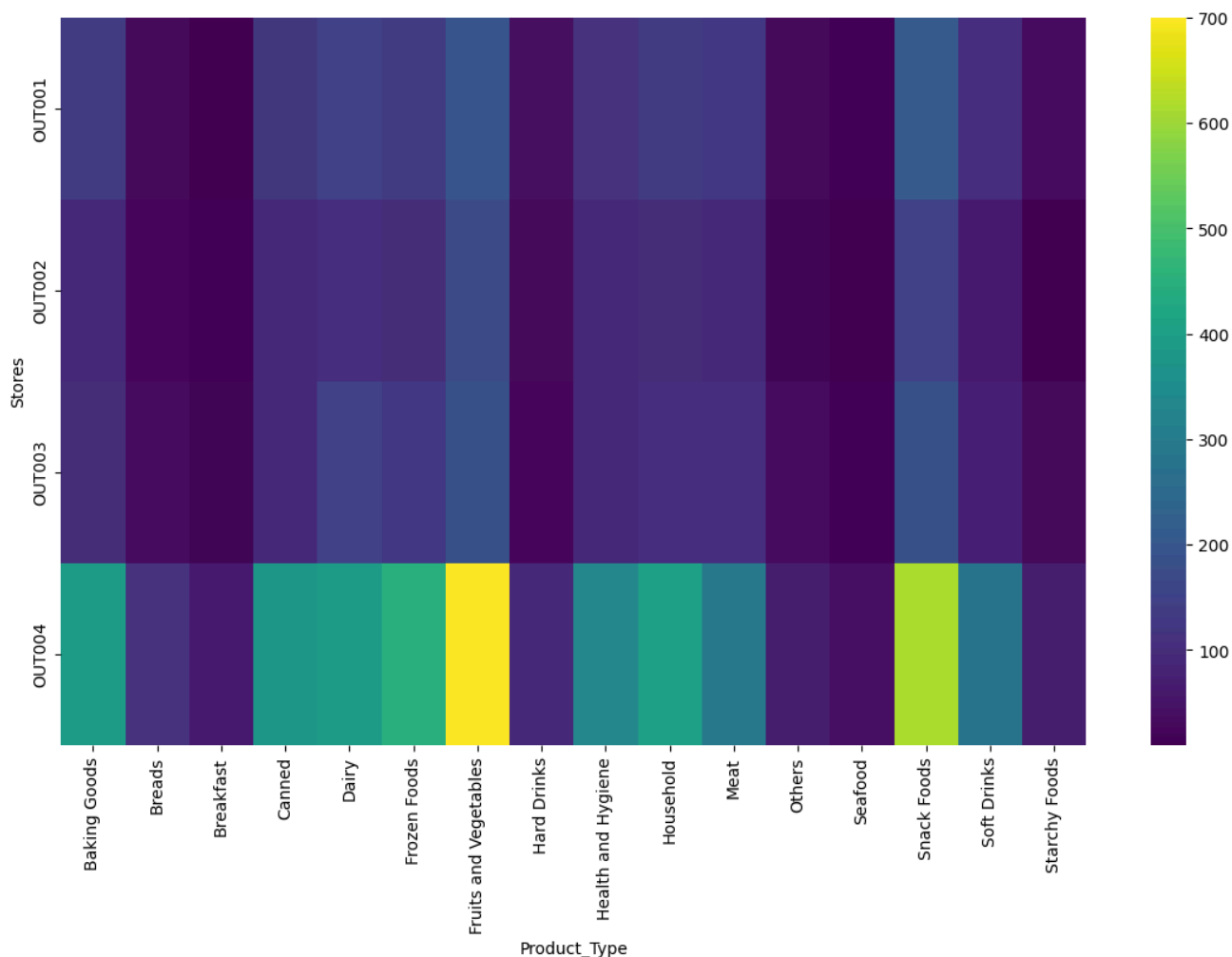


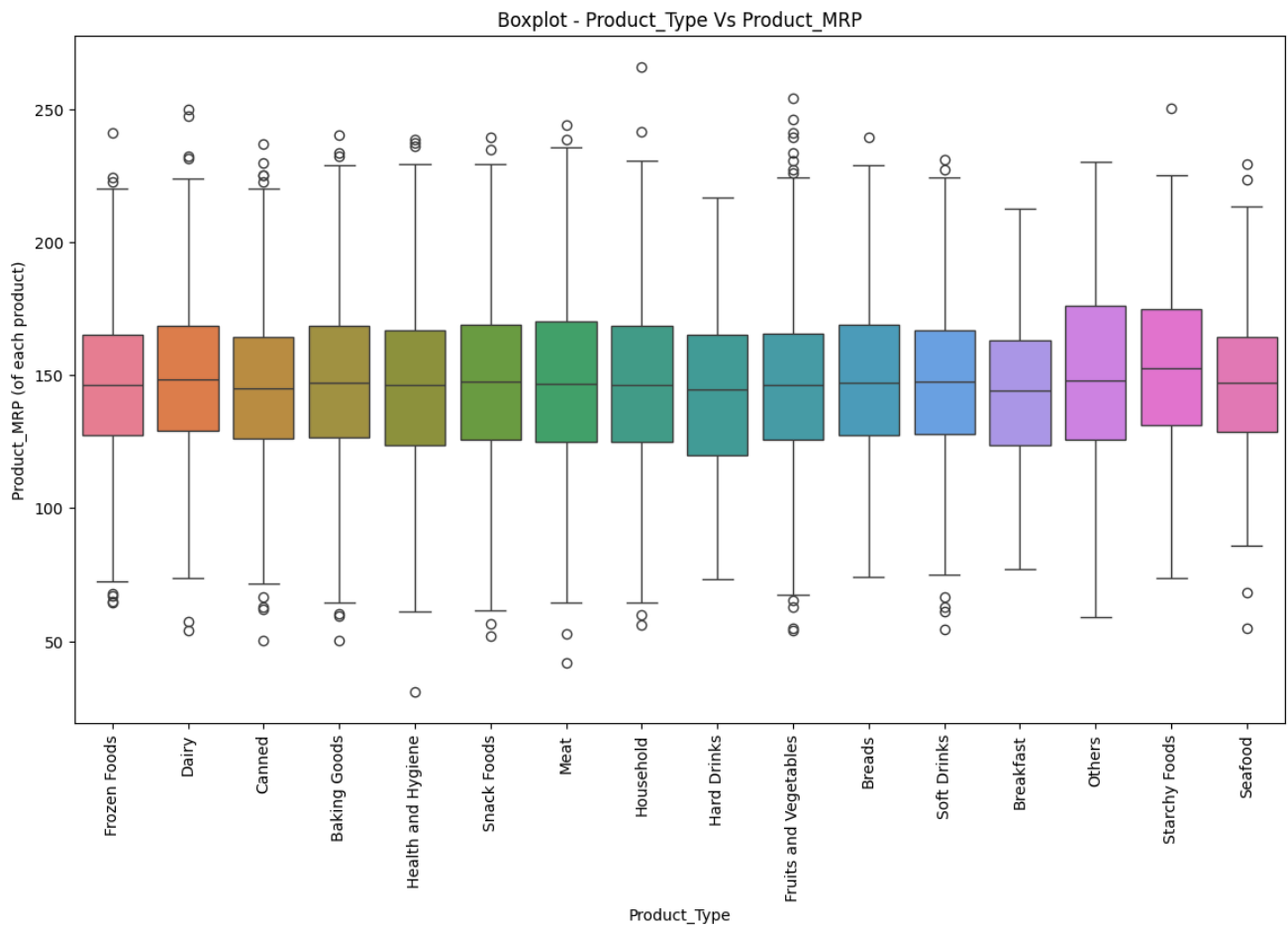


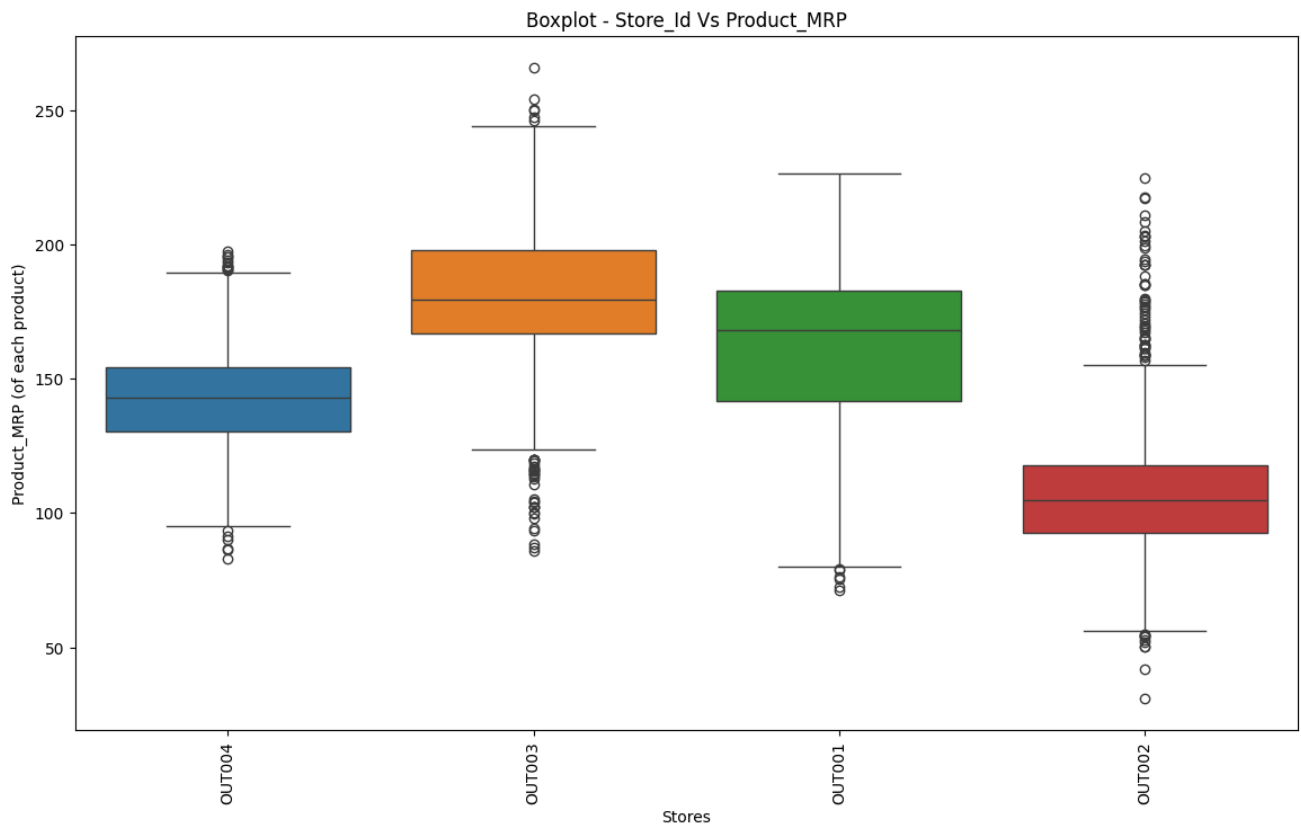












Observation : The median Product_MRP is nearly identical across all stores, however

Let's delve deeper and do a detailed analysis of each of the stores.

OUT001

```
data.loc[data["Store_Id"] == "OUT001"].describe(include="all").T
```

	count	unique	top	freq	mean	std	min	max
Product_Id	1586	1586	NC7187	1	NaN	NaN	NaN	NaN
Product_Weight	1586.0	NaN	NaN	NaN	13.458865	2.064975	6.0	18.0
Product_Sugar_Content	1586	4	Low Sugar	845	NaN	NaN	NaN	NaN
Product_Allocated_Area	1586.0	NaN	NaN	NaN	0.068768	0.047131	0.0	0.2
Product_Type	1586	16	Snack Foods	202	NaN	NaN	NaN	NaN
Product_MRP	1586.0	NaN	NaN	NaN	160.514054	30.359059	71.0	250.0
Store_Id	1586	1	OUT001	1586	NaN	NaN	NaN	NaN
Store_Establishment_Year	1586.0	NaN	NaN	NaN	1987.0	0.0	1987.0	1987.0
Store_Size	1586	1	High	1586	NaN	NaN	NaN	NaN
Store_Location_City_Type	1586	1	Tier 2	1586	NaN	NaN	NaN	NaN
Store_Type	1586	1	Supermarket Type1	1586	NaN	NaN	NaN	NaN
Product_Store_Sales_Total	1586.0	NaN	NaN	NaN	3923.778802	904.62901	2300.0	5000.0

Observations

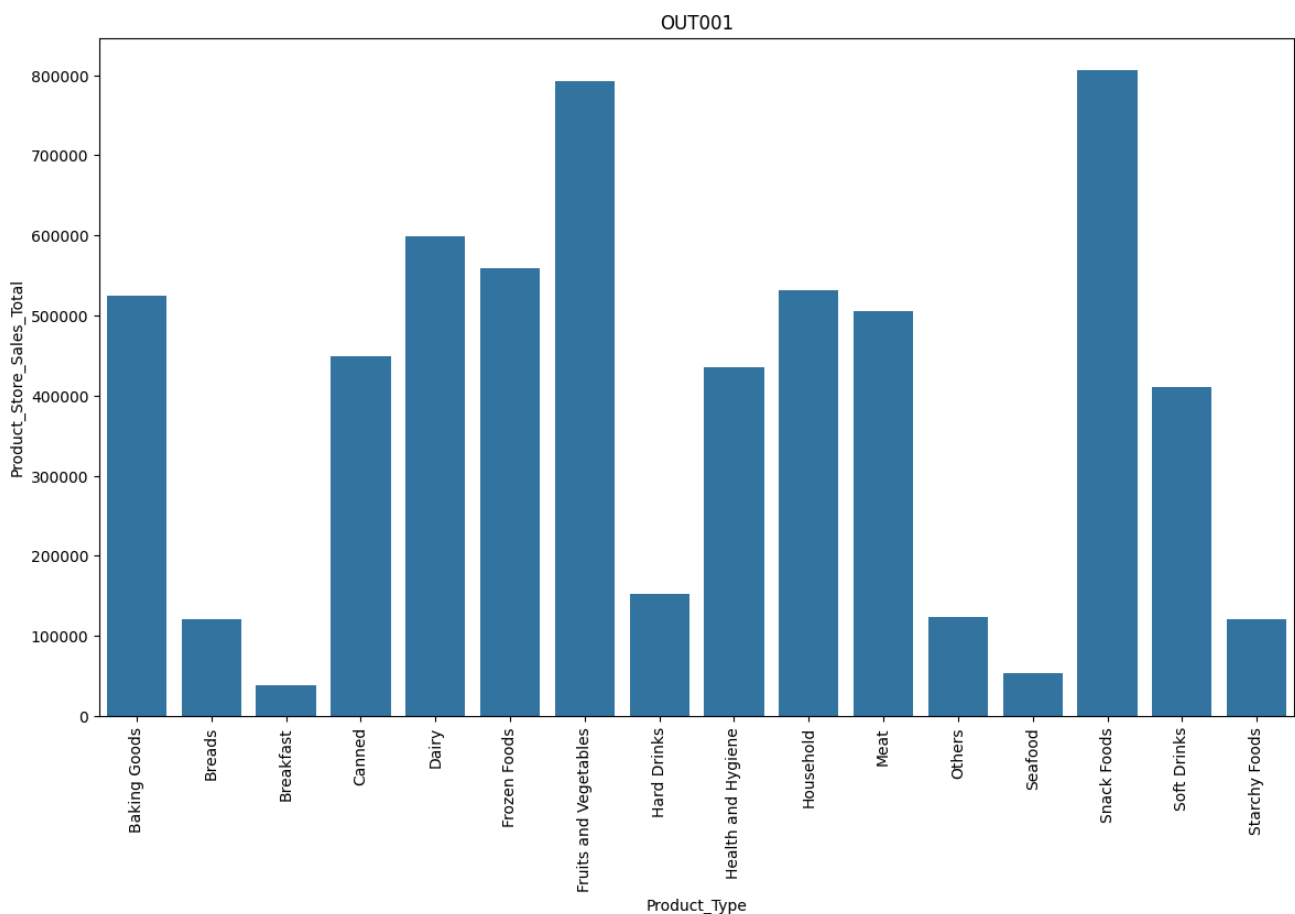
- OUT001 is a store of Supermarket Type 1 which is located in a Tier 2 city and has store size as high. It was established in 1987.
- OUT001 has sold products whose MRP range from 71 to 227.
- Snack Foods have been sold the highest number of times in OUT001.
- The revenue generated from each product at OUT001 ranges from 2300 to 5000.

```
data.loc[data["Store_Id"] == "OUT001", "Product_Store_Sales_Total"].sum()
```

```
np.float64(6223113.18)
```

OUT001 has generated total revenue of 6223113 from the sales of goods.

```
df_OUT001 = (  
    data.loc[data["Store_Id"] == "OUT001"]  
    .groupby(["Product_Type"], as_index=False)["Product_Store_Sales_Total"]  
    .sum()  
)  
plt.figure(figsize=[14, 8])  
plt.xticks(rotation=90)  
plt.xlabel("Product_Type")  
plt.ylabel("Product_Store_Sales_Total")  
plt.title("OUT001")  
sns.barplot(x=df_OUT001.Product_Type, y=df_OUT001.Product_Store_Sales_Total)  
plt.show()
```



- OUT001 has generated the highest revenue from the sale of fruits and vegetables and snack foods. Both the categories have contributed around 800000 each.

OUT002

```
data.loc[data["Store_Id"] == "OUT002"].describe(include="all").T
```

	count	unique	top	freq	mean	std	min
Product_Id	1152	1152	NC2769	1	NaN	NaN	NaN
Product_Weight	1152.0	NaN	NaN	NaN	9.911241	1.799846	4.0
Product_Sugar_Content	1152	4	Low Sugar	658	NaN	NaN	NaN
Product_Allocated_Area	1152.0	NaN	NaN	NaN	0.067747	0.047567	0.006
Product_Type	1152	16	Fruits and Vegetables	168	NaN	NaN	NaN
Product_MRP	1152.0	NaN	NaN	NaN	107.080634	24.912333	31.0
Store_Id	1152	1	OUT002	1152	NaN	NaN	NaN
Store_Establishment_Year	1152.0	NaN	NaN	NaN	1998.0	0.0	1998.0
Store_Size	1152	1	Small	1152	NaN	NaN	NaN
Store_Location_City_Type	1152	1	Tier 3	1152	NaN	NaN	NaN
Store_Type	1152	1	Food Mart	1152	NaN	NaN	NaN
Product_Store_Sales_Total	1152.0	NaN	NaN	NaN	1762.942465	462.862431	33.0

Observations

- OUT002 is a food mart which is located in a Tier 3 city and has store size as small. It was established in 1998.
- OUT002 has sold products whose MRP range from 31 to 225.
- Fruits and vegetables have been sold the highest number of times in OUT002.
- The revenue generated from each product at OUT002 ranges from 33 to 2300.

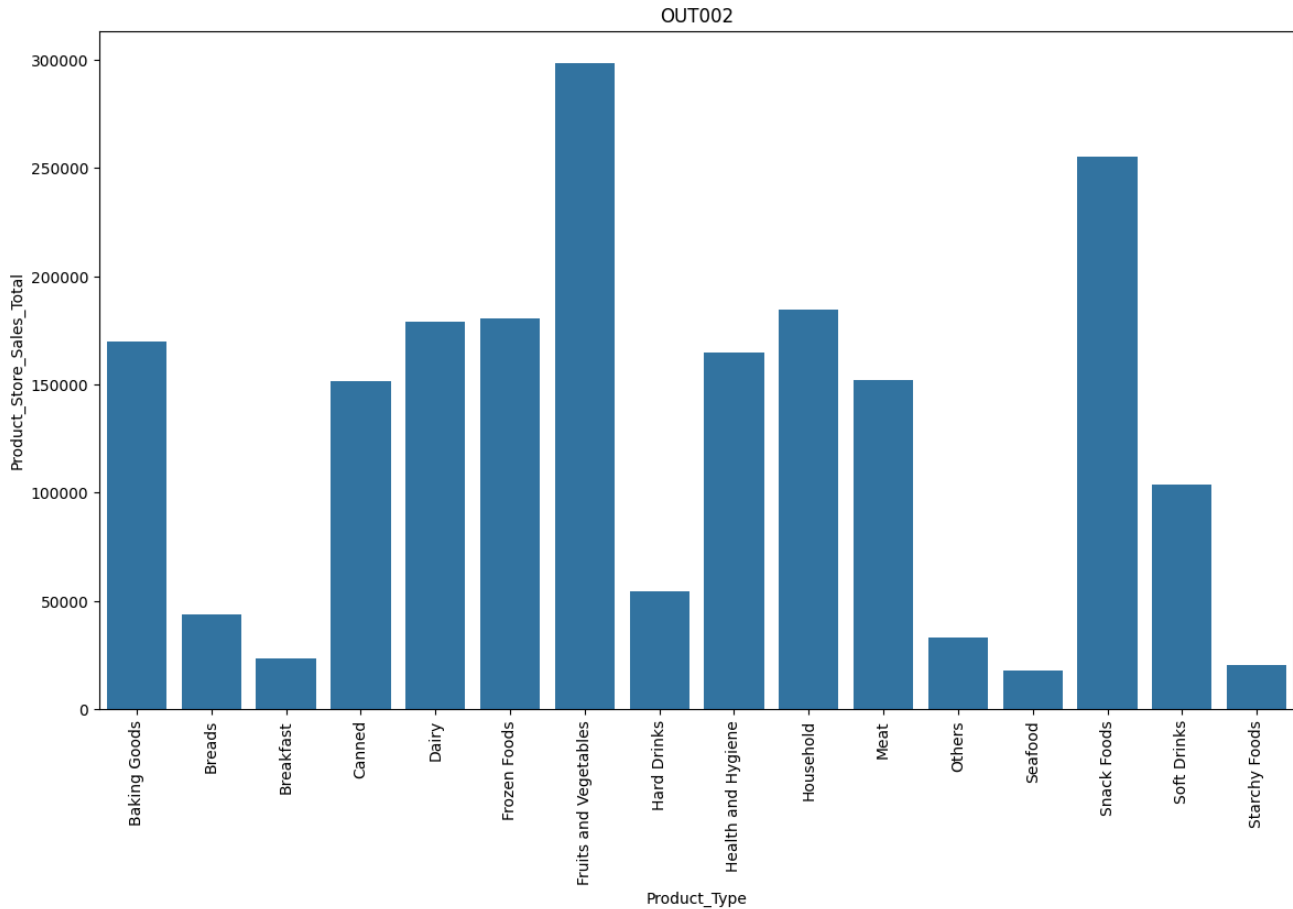
```
data.loc[data["Store_Id"] == "OUT002", "Product_Store_Sales_Total"].sum()
```

```
np.float64(2030909.72)
```

OUT002 has generated total revenue of 2030910 from the sales of goods.

```
df_OUT002 = (
    data.loc[data["Store_Id"] == "OUT002"]
    .groupby(["Product_Type"], as_index=False) ["Product_Store_Sales_Total"]
    .sum()
)
plt.figure(figsize=[14, 8])
plt.xticks(rotation=90)
plt.xlabel("Product_Type")
plt.ylabel("Product_Store_Sales_Total")
```

```
plt.title("OUT002")
sns.barplot(x=df_OUT002.Product_Type, y=df_OUT002.Product_Store_Sales_Total)
plt.show()
```



- OUT002 has generated the highest revenue from the sale of fruits and vegetables (~ 300000) followed by snack foods (~ 250000).

OUT003

```
data.loc[data["Store_Id"] == "OUT003"].describe(include="all").T
```

	count	unique	top	freq	mean	std	
Product_Id	1349	1349	NC522	1	NaN	NaN	
Product_Weight	1349.0	NaN	NaN	NaN	15.103692	1.893531	
Product_Sugar_Content	1349	4	Low Sugar	750	NaN	NaN	
Product_Allocated_Area	1349.0	NaN	NaN	NaN	0.068637	0.048708	0
Product_Type	1349	16	Snack Foods	186	NaN	NaN	
Product_MRP	1349.0	NaN	NaN	NaN	181.358725	24.796429	8
Store_Id	1349	1	OUT003	1349	NaN	NaN	
Store_Establishment_Year	1349.0	NaN	NaN	NaN	1999.0	0.0	19
Store_Size	1349	1	Medium	1349	NaN	NaN	
Store_Location_City_Type	1349	1	Tier 1	1349	NaN	NaN	
Store_Type	1349	1	Departmental Store	1349	NaN	NaN	
Product_Store_Sales_Total	1349.0	NaN	NaN	NaN	4946.966323	677.539953	306

Observations

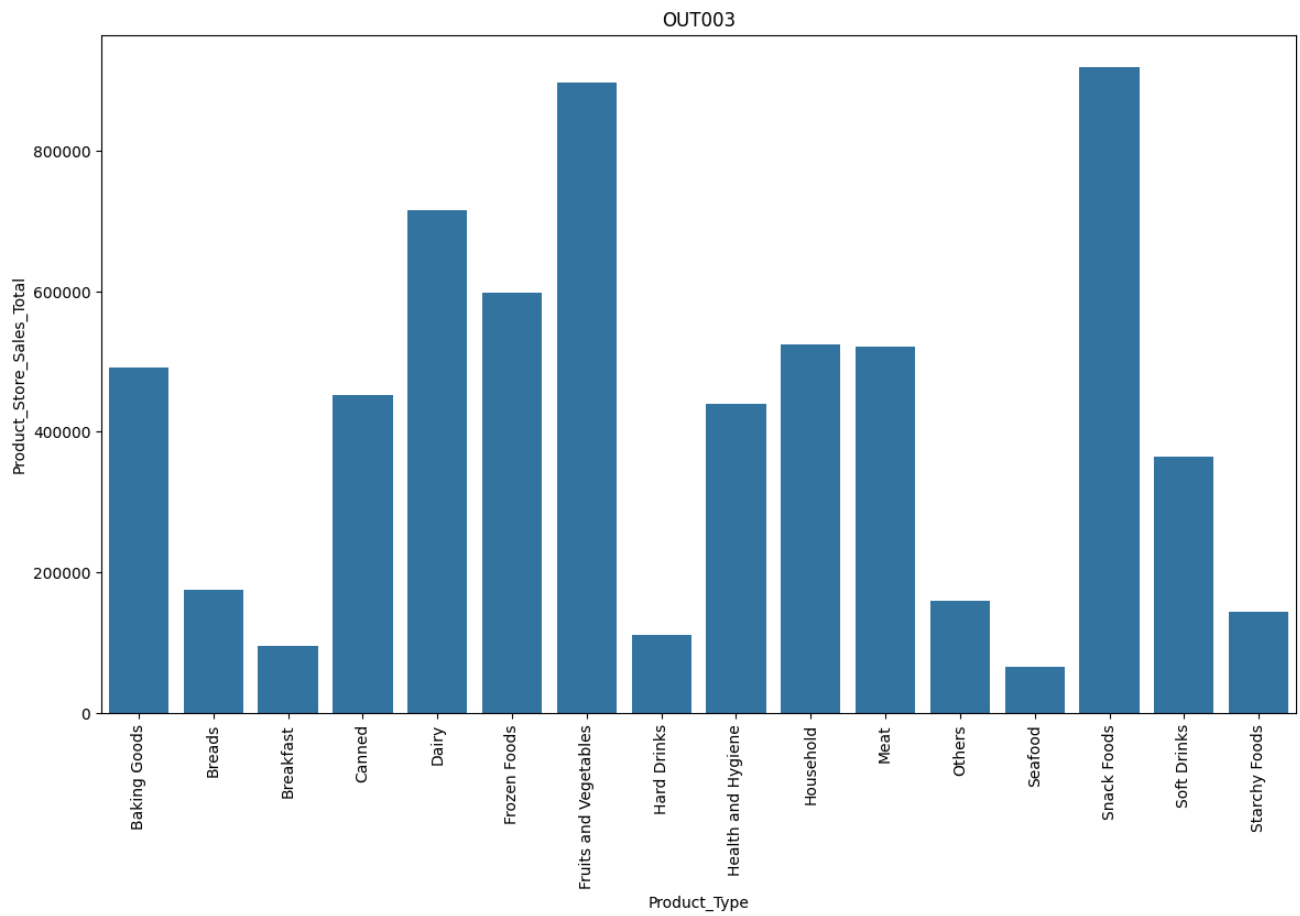
- OUT003 is a Departmental store which is located in a Tier 1 city and has store size as medium. It was established in 1999.
- OUT003 has sold products whose MRP range from 86 to 266.
- Snack Foods have been sold the highest number of times in OUT003.
- The revenue generated from each product at OUT003 ranges from 3070 to 8000.

```
data.loc[data["Store_Id"] == "OUT003", "Product_Store_Sales_Total"].sum()

np.float64(6673457.57)
```

OUT003 has generated total revenue of 6673458 from the sales of goods.

```
df_OUT003 = (
    data.loc[data["Store_Id"] == "OUT003"]
    .groupby(["Product_Type"], as_index=False)["Product_Store_Sales_Total"]
    .sum()
)
plt.figure(figsize=[14, 8])
plt.xticks(rotation=90)
plt.xlabel("Product_Type")
plt.ylabel("Product_Store_Sales_Total")
plt.title("OUT003")
sns.barplot(x=df_OUT003.Product_Type, y=df_OUT003.Product_Store_Sales_Total)
plt.show()
```



- OUT003 has generated the highest revenue from the sale of snack foods followed by fruits and vegetables, both the categories contributing around 800000 each.

OUT004

```
data.loc[data["Store_Id"] == "OUT004"].describe(include="all").T
```

	count	unique	top	freq	mean	std	m
Product_Id	4676	4676	NC584	1	NaN	NaN	NaN
Product_Weight	4676.0	NaN	NaN	NaN	12.349613	1.428199	7.3
Product_Sugar_Content	4676	4	Low Sugar	2632	NaN	NaN	NaN
Product_Allocated_Area	4676.0	NaN	NaN	NaN	0.069092	0.048584	0.00
Product_Type	4676	16	Fruits and Vegetables	700	NaN	NaN	NaN
Product_MRP	4676.0	NaN	NaN	NaN	142.399709	17.513973	83.0
Store_Id	4676	1	OUT004	4676	NaN	NaN	NaN
Store_Establishment_Year	4676.0	NaN	NaN	NaN	2009.0	0.0	2009
Store_Size	4676	1	Medium	4676	NaN	NaN	NaN
Store_Location_City_Type	4676	1	Tier 2	4676	NaN	NaN	NaN
Store_Type	4676	1	Supermarket Type2	4676	NaN	NaN	NaN
Product_Store_Sales_Total	4676.0	NaN	NaN	NaN	3299.312111	468.271692	1561.0

Observations

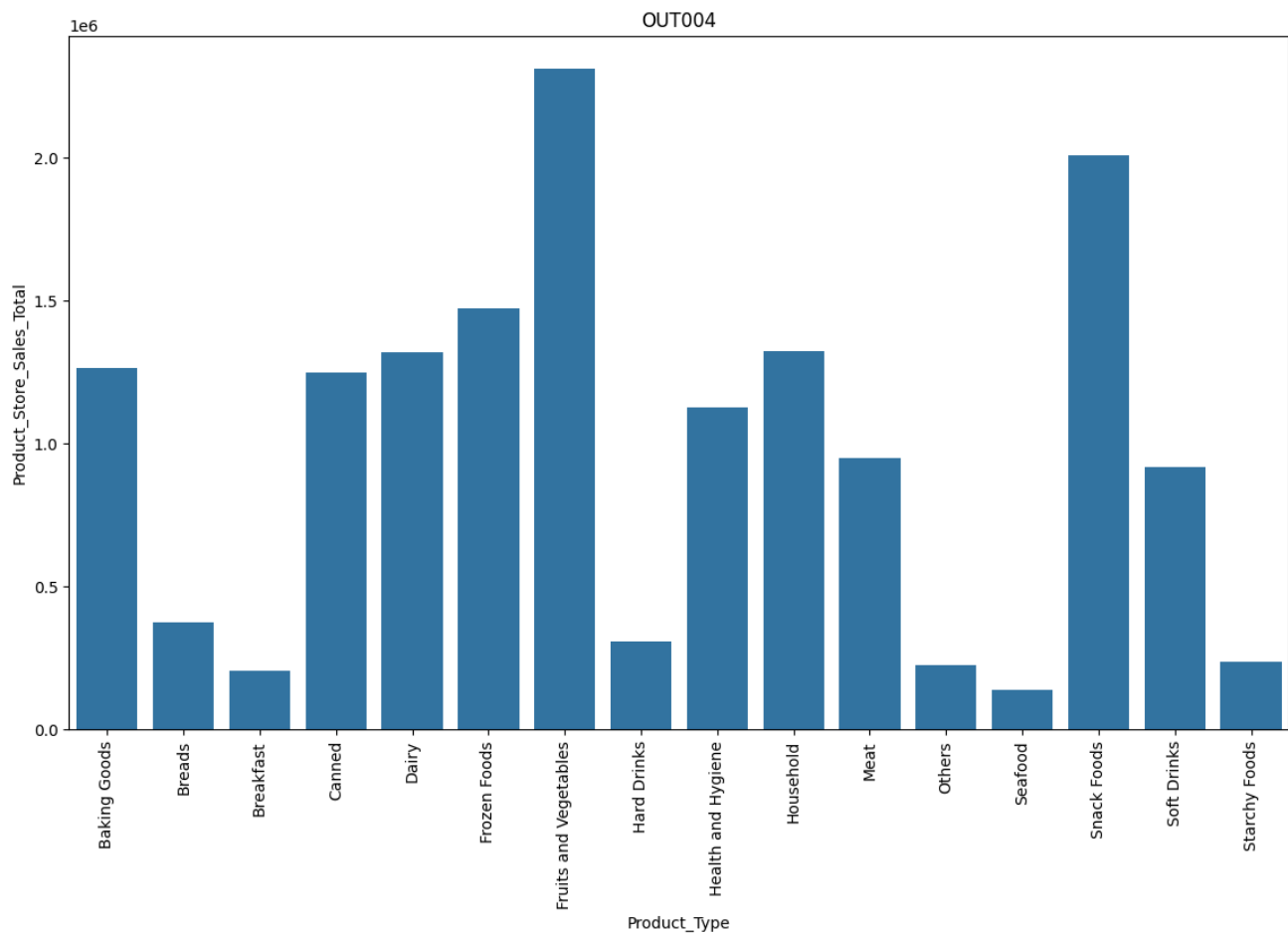
- OUT004 is a store of Supermarket Type2 which is located in a Tier 2 city and has store size as medium. It was established in 2009.
- OUT004 has sold products whose MRP range from 83 to 198.
- Fruits and vegetables have been sold the highest number of times in OUT004.
- The revenue generated from each product at OUT004 ranges from 1561 to 5463.

```
data.loc[data["Store_Id"] == "OUT004", "Product_Store_Sales_Total"].sum()
```

```
np.float64(15427583.43)
```

OUT004 has generated total revenue of 15427583 from the sales of goods which is highest among all the 4 stores.

```
df_OUT004 = (
    data.loc[data["Store_Id"] == "OUT004"]
    .groupby(["Product_Type"], as_index=False) ["Product_Store_Sales_Total"]
    .sum()
)
plt.figure(figsize=[14, 8])
plt.xticks(rotation=90)
plt.xlabel("Product_Type")
plt.ylabel("Product_Store_Sales_Total")
plt.title("OUT004")
sns.barplot(x=df_OUT004.Product_Type, y=df_OUT004.Product_Store_Sales_Total)
plt.show()
```



- OUT004 has generated the highest revenue from the sale of fruits and vegetables (~ 2500000) followed by snack foods (~ 2000000).

Let's find out the revenue generated by the stores from each of the product types.

```
df1 = data.groupby(["Product_Type", "Store_Id"], as_index=False)[  
    "Product_Store_Sales_Total"  
].sum()  
df1
```


	Product_Type	Store_Id	Product_Store_Sales_Total
0	Baking Goods	OUT001	525131.04
1	Baking Goods	OUT002	169860.50
2	Baking Goods	OUT003	491908.20
3	Baking Goods	OUT004	1266086.26
4	Breads	OUT001	121274.09
5	Breads	OUT002	43419.47
6	Breads	OUT003	175391.93
7	Breads	OUT004	374856.75
8	Breakfast	OUT001	38161.10
9	Breakfast	OUT002	23396.10
10	Breakfast	OUT003	95634.08
11	Breakfast	OUT004	204939.13
12	Canned	OUT001	449016.38
13	Canned	OUT002	151467.66
14	Canned	OUT003	452445.17
15	Canned	OUT004	1247153.50
16	Dairy	OUT001	598767.62
17	Dairy	OUT002	178888.18
18	Dairy	OUT003	715814.94
19	Dairy	OUT004	1318447.30
20	Frozen Foods	OUT001	558556.81
21	Frozen Foods	OUT002	180295.95
22	Frozen Foods	OUT003	597608.42
23	Frozen Foods	OUT004	1473519.65
24	Fruits and Vegetables	OUT001	792992.59
25	Fruits and Vegetables	OUT002	298503.56
26	Fruits and Vegetables	OUT003	897437.46
27	Fruits and Vegetables	OUT004	2311899.66
28	Hard Drinks	OUT001	152920.74
29	Hard Drinks	OUT002	54281.85
30	Hard Drinks	OUT003	110760.30
31	Hard Drinks	OUT004	307851.73
32	Health and Hygiene	OUT001	435005.31
33	Health and Hygiene	OUT002	164660.81

	Product_Type	Store_Id	Product_Store_Sales_Total
34	Health and Hygiene	OUT003	439139.18
35	Health and Hygiene	OUT004	1124901.91
36	Household	OUT001	531371.38
37	Household	OUT002	184665.65
38	Household	OUT003	523981.64
39	Household	OUT004	1324721.50
40	Meat	OUT001	505867.28
41	Meat	OUT002	151800.01
42	Meat	OUT003	520939.68
43	Meat	OUT004	950604.97
44	Others	OUT001	123977.09
45	Others	OUT002	32835.73
46	Others	OUT003	159963.75
47	Others	OUT004	224719.73
48	Seafood	OUT001	52936.84
49	Seafood	OUT002	17663.35
50	Seafood	OUT003	65337.48
51	Seafood	OUT004	136466.37
52	Snack Foods	OUT001	806142.24
53	Snack Foods	OUT002	255317.57
54	Snack Foods	OUT003	918510.44
55	Snack Foods	OUT004	2009026.70
56	Soft Drinks	OUT001	410548.69
57	Soft Drinks	OUT002	103808.35
58	Soft Drinks	OUT003	365046.30
59	Soft Drinks	OUT004	917641.38
60	Starchy Foods	OUT001	120443.98
61	Starchy Foods	OUT002	20044.98
62	Starchy Foods	OUT003	143538.60
63	Starchy Foods	OUT004	234746.89

- In all the product types, the revenue generated by OUT004 has been the highest which seems quite logical since around 53% of the total products were brought from this store.

- In all the product categories, the revenue generated by OUT002 has been the lowest which seems quite obvious since it is small store in a Tier 3 city.

Let's find out the revenue generated by the stores from products having different levels of sugar content.

```
df2 = data.groupby(["Product_Sugar_Content", "Store_Id"], as_index=False) [
    "Product_Store_Sales_Total"
].sum()
df2
```

	Product_Sugar_Content	Store_Id	Product_Store_Sales_Total
0	Low Sugar	OUT001	3300834.93
1	Low Sugar	OUT002	1156758.85
2	Low Sugar	OUT003	3706903.24
3	Low Sugar	OUT004	8658908.78
4	No Sugar	OUT001	1090353.78
5	No Sugar	OUT002	382162.19
6	No Sugar	OUT003	1123084.57
7	No Sugar	OUT004	2674343.14
8	Regular	OUT001	1749444.51
9	Regular	OUT002	472112.50
10	Regular	OUT003	1743566.35
11	Regular	OUT004	3902547.93
12	reg	OUT001	82479.96
13	reg	OUT002	19876.18
14	reg	OUT003	99903.41
15	reg	OUT004	191783.58

- The trend is the same as that which was present in the revenue analysis of stores for product types.

Data Preprocessing

Replacing the values in the Product_Sugar_Content column

We can observe that in the Product_Sugar_Content column, there are 3 types - Low Sugar, Regular and reg.

It seems quite obvious that Regular and reg are referring to the same category. So let's replace reg with Regular.

```
data.Product_Sugar_Content.value_counts()
```

	count
Product_Sugar_Content	
Low Sugar	4885
Regular	2251
No Sugar	1519
reg	108

dtype: int64

```
# Observation : The count for Regular sugar is 2251 and for reg is 108
```

```
# Replacing reg with Regular
```

```
data.Product_Sugar_Content.replace(to_replace=["reg"], value=["Regular"], inplace=True)
```

```
data.Product_Sugar_Content.value_counts()
```

	count
Product_Sugar_Content	
Low Sugar	4885
Regular	2359
No Sugar	1519

dtype: int64

```
#Observation: After replacing or merging reg wiht Regular the count of regular is
```

Exploring Patterns in Product_IDs

We can see that the Product_Id column has two characters followed by a number.

Let's delve deeper and see whether they are having any relationship with the other columns or not

```
## extracting the first two characters from the Product_Id column and storing it in  
data["Product_Id_char"] = data["Product_Id"].str[:2]  
data.head()
```

	Product_Id	Product_Weight	Product_Sugar_Content	Product_Allocated_Area	Product_Typ
0	FD6114	12.66	Low Sugar	0.027	Frozen Foo
1	FD7839	16.54	Low Sugar	0.144	Dair
2	FD5075	14.28	Regular	0.031	Canne
3	FD8233	12.10	Low Sugar	0.112	Baking Goo
4	NC1180	9.57	No Sugar	0.010	Health an Hygier

```
data["Product_Id_char"].unique()
```

```
array(['FD', 'NC', 'DR'], dtype=object)
```

Observation : There are 3 different types of product id – FD –Food , NC (Non Consumable), DR (Drinks)

```
data.loc[data.Product_Id_char == "FD", "Product_Type"].unique()
```

```
array(['Frozen Foods', 'Dairy', 'Canned', 'Baking Goods', 'Snack Foods',  
      'Meat', 'Fruits and Vegetables', 'Breads', 'Breakfast',  
      'Starchy Foods', 'Seafood'], dtype=object)
```

Observation : There are 11 sub categories under product id – FD –Food.

```
data.loc[data.Product_Id_char == "NC", "Product_Type"].unique()
```

```
array(['Health and Hygiene', 'Household', 'Others'], dtype=object)
```

Observation : There are 2 sub categories under product id – NC (Non Consumable)

```
data.loc[data.Product_Id_char == "DR", "Product_Type"].unique()
```

```
array(['Hard Drinks', 'Soft Drinks'], dtype=object)
```

Observation : There are 2 sub categories under product id – DR (Drinks)

Store's Age

A store which has been in the business for a long duration is more trustworthy than the newly established ones.

On the other hand, older stores may sometimes lack infrastructure if proper attention is not given. So let us calculate the current age of the store and incorporate that in our model.

```
# Outlet Age
```

```
data["Store_Age_Years"] = 2025 - data.Store_Establishment_Year
```

```
print(data.Store_Age_Years.min())
```

```
print(data.Store_Age_Years.max())
```

```
16
38
```

Grouping Product Types into Perishables and Non-Perishables.

We have 16 different product types in our dataset.

So let us make two broad categories, perishables and non perishables, in order to reduce the number of product types

```
perishables = [
    "Dairy",
    "Meat",
    "Fruits and Vegetables",
    "Breakfast",
    "Breads",
    "Seafood",
]
```

```
def change(x):
    if x in perishables:
        return "Perishables"
    else:
        return "Non Perishables"
```

```
data['Product_Type_Category'] = data['Product_Type'].apply(change)
```

```
data.head()
```

	Product_Id	Product_Weight	Product_Sugar_Content	Product_Allocated_Area	Product_Type
0	FD6114	12.66	Low Sugar	0.027	Frozen Food
1	FD7839	16.54	Low Sugar	0.144	Dairy
2	FD5075	14.28	Regular	0.031	Canned
3	FD8233	12.10	Low Sugar	0.112	Baking Goods
4	NC1180	9.57	No Sugar	0.010	Health and Hygiene

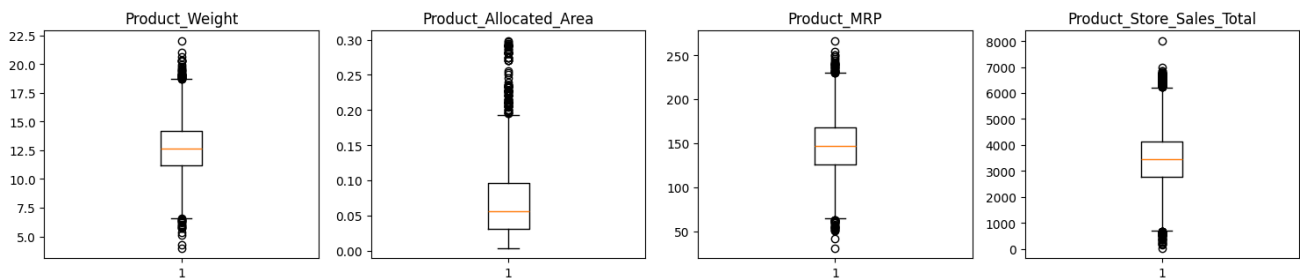
Outlier Check

```
# outlier detection using boxplot
numeric_columns = data.select_dtypes(include=np.number).columns.tolist()
numeric_columns.remove("Store_Establishment_Year")
numeric_columns.remove("Store_Age_Years")
```

```
plt.figure(figsize=(15, 12))

for i, variable in enumerate(numeric_columns):
    plt.subplot(4, 4, i + 1)
    plt.boxplot(data[variable], whis=1.5)
    plt.tight_layout()
    plt.title(variable)

plt.show()
```



#Observaton : Product_Allocated_Area and Product_Store_Sales_Total show a heavy correlation

Data Preparation for Modeling

- We aim to forecast the `Product_Store_Sales_Total`.
- Before building the model, we'll drop unnecessary columns and encode the categorical features.
- We'll then split the data into training and testing sets to evaluate the model's performance on unseen data.

```
data.head()
```

	Product_Id	Product_Weight	Product_Sugar_Content	Product_Allocated_Area	Product_Type
0	FD6114	12.66	Low Sugar	0.027	Frozen Food
1	FD7839	16.54	Low Sugar	0.144	Dairy
2	FD5075	14.28	Regular	0.031	Canned
3	FD8233	12.10	Low Sugar	0.112	Baking Goods
4	NC1180	9.57	No Sugar	0.010	Health and Hygiene

Let's remove the columns that are not required.

```
data = data.drop(["Product_Id", "Product_Type", "Store_Id", "Store_Establishment_Year", "Store_Year"])
```

```
data.shape
```

```
(8763, 11)
```

```
data.head()
```

	Product_Weight	Product_Sugar_Content	Product_Allocated_Area	Product_MRP	Store_Size
0	12.66	Low Sugar	0.027	117.08	Medium
1	16.54	Low Sugar	0.144	171.43	Medium
2	14.28	Regular	0.031	162.08	High
3	12.10	Low Sugar	0.112	186.31	High
4	9.57	No Sugar	0.010	123.67	Small

```
# Separating features and the target column
X = data.drop("Product_Store_Sales_Total", axis=1)
y = data["Product_Store_Sales_Total"]
```

```
# Splitting the data into train and test sets in 70:30 ratio
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1, shuffle=True #Complete the code to define
)
```

```
X_train.shape, X_test.shape
```

```
((6134, 10), (2629, 10))
```

Data Pre-processing Pipeline

```
categorical_features = data.select_dtypes(include=['object', 'category']).columns.tolist()
categorical_features
```

```
['Product_Sugar_Content',
 'Store_Size',
 'Store_Location_City_Type',
 'Store_Type',
 'Product_Id_char',
 'Product_Type_Category']
```

```
# Create a preprocessing pipeline for the categorical features
```

```
preprocessor = make_column_transformer(
    (Pipeline([('encoder', OneHotEncoder(handle_unknown='ignore'))]), categorical_features),
    remainder='passthrough'
)
```

Model Building

Define functions for Model Evaluation

```
# function to compute adjusted R-squared
def adj_r2_score(predictors, targets, predictions):
    r2 = r2_score(targets, predictions)
    n = predictors.shape[0]
    k = predictors.shape[1]
    return 1 - ((1 - r2) * (n - 1) / (n - k - 1))

# function to compute different metrics to check performance of a regression model
def model_performance_regression(model, predictors, target):
    """
    Function to compute different metrics to check regression model performance

    model: regressor
    predictors: independent variables
    target: dependent variable
    """

    # predicting using the independent variables
    pred = model.predict(predictors)

    r2 = r2_score(target, pred) # to compute R-squared
    adjr2 = adj_r2_score(predictors, target, pred) # to compute adjusted R-squared
    rmse = np.sqrt(mean_squared_error(target, pred)) # to compute RMSE
    mae = mean_absolute_error(target, pred) # to compute MAE
    mape = mean_absolute_percentage_error(target, pred) # to compute MAPE

    # creating a dataframe of metrics
    df_perf = pd.DataFrame(
        {
            "RMSE": rmse,
            "MAE": mae,
            "R-squared": r2,
            "Adj. R-squared": adjr2,
            "MAPE": mape,
        },
        index=[0],
    )

    return df_perf
```

The ML models to be built can be any two out of the following:

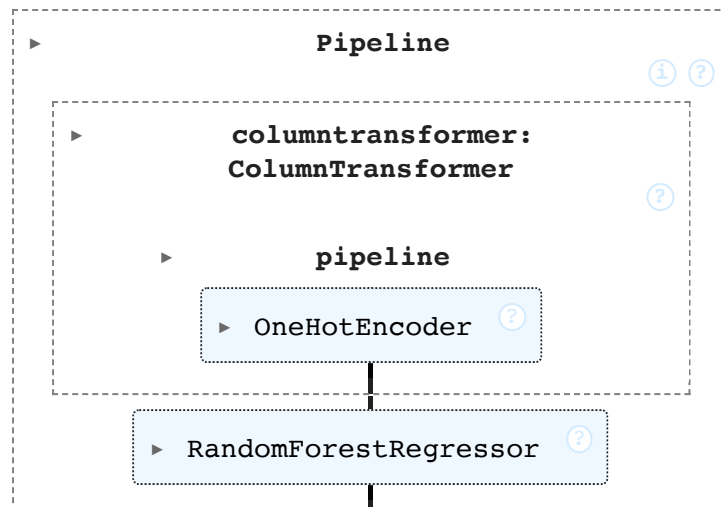
1. Decision Tree
2. Bagging
3. Random Forest
4. AdaBoost
5. Gradient Boosting
6. XGBoost

```
# Chosing Random Forest and XGBoost for this project
```

Random Forest Model

```
# random forest regressor is to be used

rf_estimator = RandomForestRegressor(random_state=1)
rf_estimator = make_pipeline(preprocessor, rf_estimator)
rf_estimator.fit(X_train, y_train)
```



Checking model performance on training set

```
rf_estimator_model_train_perf = model_performance_regression(rf_estimator, X_train, y_train)
rf_estimator_model_train_perf
```

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	596.994959	468.87585	0.685016	0.684501	0.165674

Observation :

An R-squared of 0.685 indicates that the model explains approximately 68.5% of the variance in the target variable.
The RMSE of 596.99 and MAE of 468.88 suggest that, on average, the model's sales predictions are off by approximately 597 and 469 units, respectively.
A MAPE (Mean Absolute Percentage Error) of 0.165 means the model's predictions are off by approximately 16.5% on average.

Checking model performance on test set

```
rf_estimator_model_test_perf = model_performance_regression(rf_estimator, X_test, y_test)
rf_estimator_model_test_perf
```

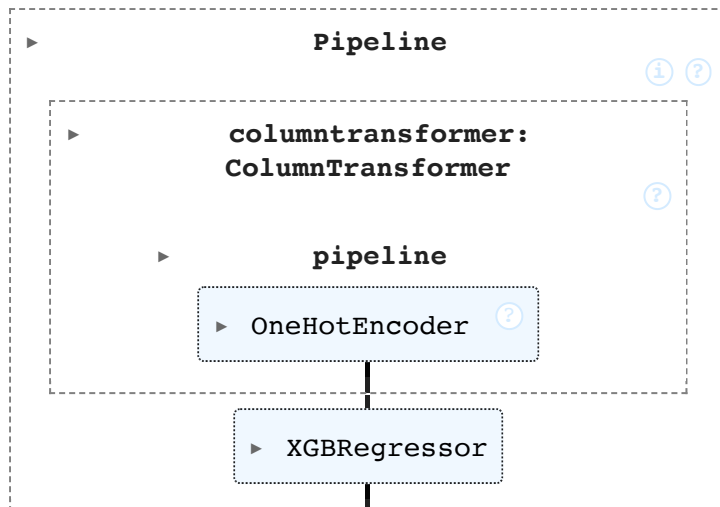
	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	615.906846	485.311027	0.66851	0.667244	0.187394

Observation :

The R-squared of 0.668 is very close to the training score (0.685), which indicates that the model is not overfitting.
The RMSE of 615.91 and MAE of 485.31 show a slight but expected increase compared to the training set, which is a good sign.
A MAPE of 0.187 suggests the model maintains an accuracy level within approximately 18.7% on the test set.

XGBoost Regressor

```
xgb_estimator = XGBRegressor(random_state=1)
xgb_estimator = make_pipeline(preprocessor, xgb_estimator)
xgb_estimator.fit(X_train, y_train)
```



Checking model performance on training set

```
xgb_estimator_model_train_perf = model_performance_regression(xgb_estimator, X_train, y_train)
xgb_estimator_model_train_perf
```

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	596.978222	468.965507	0.685033	0.684519	0.16569

#Observation :

With an R-squared of 0.6850, the XGBoost model explains approximately 68.5% of the variance in the target variable.
The RMSE of 596.98 and MAE of 468.97 indicate that the model's predictions typically differ from the actual values by around 500 units.
The MAPE of 0.1657 (16.57%) confirms that the model is performing well across the range of the target variable.

Checking model performance on test set

```
xgb_estimator_model_test_perf = model_performance_regression(xgb_estimator, X_test, y_test)
xgb_estimator_model_test_perf
```

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	615.933034	485.429585	0.668482	0.667215	0.187421

#Observation :

The R-squared of 0.6685 on the test set is nearly identical to the training score, indicating good generalization.
The RMSE of 615.93 and MAE of 485.43 indicate that the model's average error on new data is slightly higher than on the training data.
A MAPE of 18.74% means the model is, on average, within about 81% accuracy of the actual values.

Model Performance Improvement - Hyperparameter Tuning

Hyperparameter Tuning - Random Forest

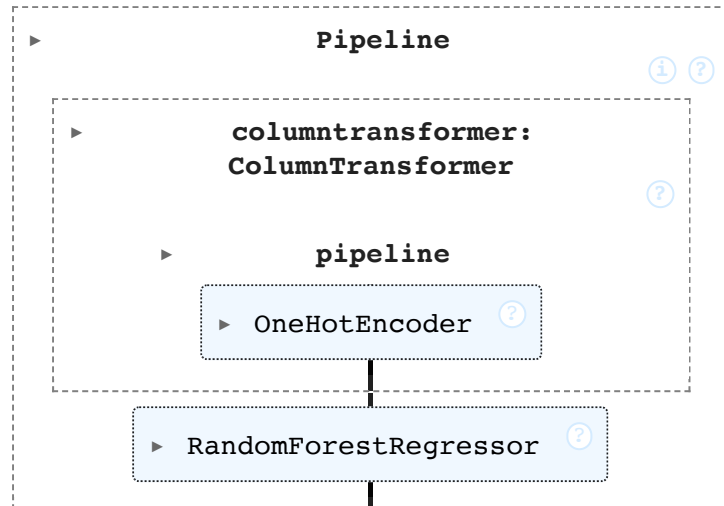
```
# # Choose the type of classifier.
rf_tuned = RandomForestRegressor(random_state=1)
rf_tuned = make_pipeline(preprocessor, rf_tuned)

# # Grid of parameters to choose from
parameters = {
    "randomforestregressor__max_depth": [5, 10, 25],
    "randomforestregressor__max_features": [0.3, 0.7, "sqrt"],
    "randomforestregressor__n_estimators": [100, 300, 500],
}

# # Run the grid search
grid_obj = GridSearchCV(rf_tuned, parameters, scoring=r2_score, cv=3, n_jobs = -1)
grid_obj = grid_obj.fit(X_train, y_train)

# # Set the clf to the best combination of parameters
rf_tuned = grid_obj.best_estimator_

# # Fit the best algorithm to the data.
rf_tuned.fit(X_train, y_train)
```



#Observation :

max_depth ([5, 10, 25]): This controls how deep the trees grow. We avoid very deep trees to prevent overfitting.
max_features ([0.3, 0.7, "sqrt"]): This tells each tree to look at 80% to 100% of the features.
n_estimators ([100, 300, 500]): More trees usually lead to a more stable model.

Checking model performance on training set

```
rf_tuned_model_train_perf = model_performance_regression(rf_tuned, X_train, y_train)
rf_tuned_model_train_perf
```

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	597.04224	469.325279	0.684966	0.684451	0.165776

```
# Observation :
# The R-squared decreased slightly to 0.6849 (from the baseline 0.6850). This indic
```

Checking model performance on test set

```
rf_tuned_model_test_perf = model_performance_regression(rf_tuned, X_test, y_test)
rf_tuned_model_test_perf
```

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	615.733194	485.552661	0.668697	0.667431	0.187437

```
# Observation :
# The R-squared increased to 0.6687 (up from 0.6685 in the baseline). While the imp
```

Hyperparameter Tuning - XGBoost Regressor

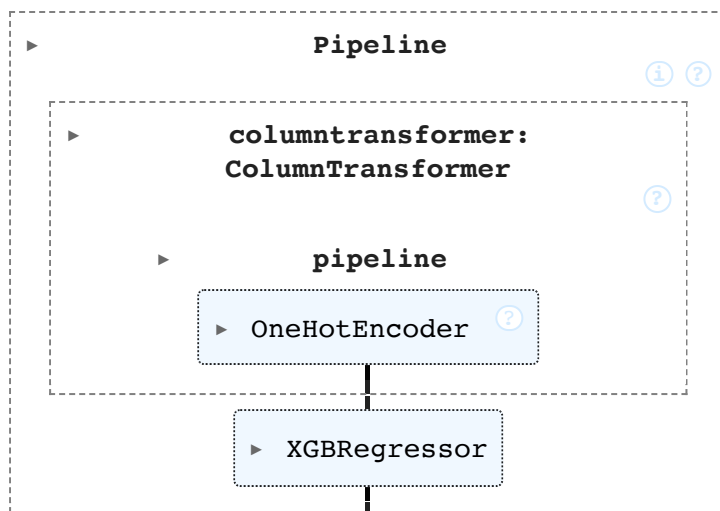
```
# # Choose the type of classifier.
xgb_tuned = XGBRegressor(random_state=1)
xgb_tuned = make_pipeline(preprocessor, xgb_tuned)

# # Grid of parameters to choose from
parameters = {
    "xgbregressor__n_estimators": [100, 150, 200],
    "xgbregressor__subsample": [0.7, 0.9, 1],
    "xgbregressor__gamma": [0, 1, 5],
    "xgbregressor__colsample_bytree": [0.7, 0.9, 1],
    "xgbregressor__colsample_bylevel": [0.7, 0.9, 1],
}

# # Run the grid search
grid_obj = GridSearchCV(xgb_tuned, parameters, scoring=r2_score, cv=3, n_jobs = -1)
grid_obj = grid_obj.fit(X_train, y_train)

# # Set the clf to the best combination of parameters
xgb_tuned = grid_obj.best_estimator_

# # Fit the best algorithm to the data.
xgb_tuned.fit(X_train, y_train)
```



```
#Observations :
# n_estimators ([100, 150, 200]): Defines the number of boosting rounds.
# subsample ([0.7, 0.9, 1]): This is the fraction of data to be randomly sampled for
# gamma ([0, 1, 5]): This is a regularization parameter.
# colsample_bytree & colsample_bylevel ([0.7, 0.9, 1]): These parameters control the
```

Checking model performance on training set

```
xgb_tuned_model_train_perf = model_performance_regression(xgb_tuned, X_train, y_train)
xgb_tuned_model_train_perf
```

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	597.071168	468.891775	0.684935	0.684421	0.165613

```
#Observations :
# The model maintains a strong R-squared of 0.6849, explaining nearly 68.5% of the
# The RMSE of 597.07 and MAE of 468.89 show that the model's predictions typically
```

Checking model performance on test set

```
xgb_tuned_model_test_perf = model_performance_regression(xgb_tuned, X_test, y_test)
xgb_tuned_model_test_perf
```

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	616.089902	485.176304	0.668313	0.667046	0.18737

```
# Observation :
# The R-squared of 0.6683 on the test set is very close to the training performance
# The RMSE of 616.09 and MAE of 485.18 indicate that the model's typical error on new
```

Model Performance Comparison, Final Model Selection, and Serialization

```
# training performance comparison

models_train_comp_df = pd.concat(
    [
        rf_estimator_model_train_perf.T,
        rf_tuned_model_train_perf.T,
        xgb_estimator_model_train_perf.T,
        xgb_tuned_model_train_perf.T,
    ],
    axis=1,
)

models_train_comp_df.columns = [
    "Random Forest",
    "Tuned Random Forest",
    "XGBoost",
    "Tuned XGBoost"
]
print("Training performance comparison:")
models_train_comp_df
```

Training performance comparison:

	Random Forest	Tuned Random Forest	XGBoost	Tuned XGBoost
RMSE	596.994959	597.042240	596.978222	597.071168
MAE	468.875850	469.325279	468.965507	468.891775
R-squared	0.685016	0.684966	0.685033	0.684935
Adj. R-squared	0.684501	0.684451	0.684519	0.684421
MAPE	0.165674	0.165776	0.165690	0.165613

```
# Observation :
# All four models (Random Forest, Tuned RF, XGBoost, and Tuned XGBoost) have conver
# The RMSE (~597) and MAE (~469) are nearly identical across the board. While the X
```

```
# test performance comparison

models_test_comp_df = pd.concat(
    [
        rf_estimator_model_test_perf.T,
        rf_tuned_model_test_perf.T,
        xgb_estimator_model_test_perf.T,
        xgb_tuned_model_test_perf.T,
    ],
    axis=1,
)

models_test_comp_df.columns = [
    "Random Forest",
    "Tuned Random Forest",
    "XGBoost",
    "Tuned XGBoost"
]
print("Test performance comparison:")
models_test_comp_df
```

Test performance comparison:

	Random Forest	Tuned Random Forest	XGBoost	Tuned XGBoost
RMSE	615.906846	615.733194	615.933034	616.089902
MAE	485.311027	485.552661	485.429585	485.176304
R-squared	0.668510	0.668697	0.668482	0.668313
Adj. R-squared	0.667244	0.667431	0.667215	0.667046
MAPE	0.187394	0.187437	0.187421	0.187370

```
# Observation : Tuned Random Forest is the final model .  
# Highest R-squared: At 0.668697, the Tuned Random Forest has the highest R-Square  
# It also achieved the lowest RMSE (615.73), meaning its predictions have the small  
  
# The difference between the best Random Forest and the best XGBoost is less than 0
```

```
# Create a folder for storing the files needed for web app deployment  
os.makedirs("backend_files", exist_ok=True)
```

```
# Define the file path to save (serialize) the trained model along with the data pr  
saved_model_path = "backend_files/tunedrandomforest_model.joblib"
```

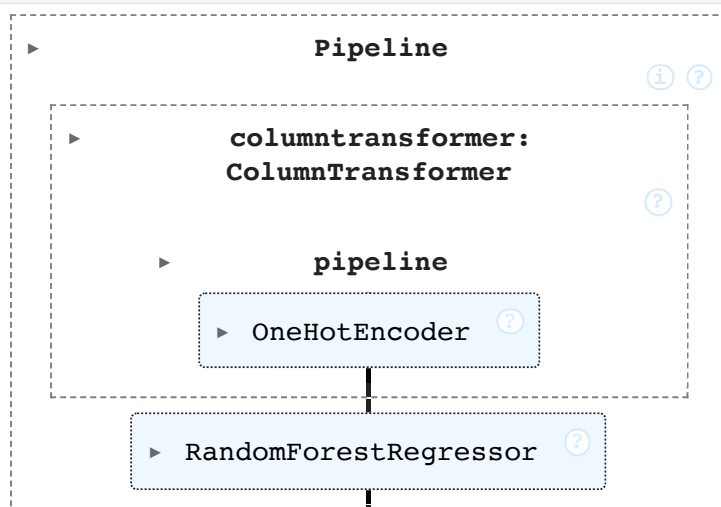
```
# Save the best trained model pipeline using joblib  
joblib.dump(rf_tuned, saved_model_path) #Complete the code to pass the variable nam  
  
print(f"Model saved successfully at {saved_model_path}")
```

Model saved successfully at backend_files/tunedrandomforest_model.joblib

```
# Load the saved model pipeline from the file  
saved_model = joblib.load("backend_files/tunedrandomforest_model.joblib") #Complete  
  
# Confirm the model is loaded  
print("Model loaded successfully.")
```

Model loaded successfully.

saved_model



Let's try making predictions on the test set using the deserialized model.

- Please ensure that the saved model is loaded before making predictions.

```
saved_model.predict(X_test) #Complete the code to pass the X_test for inference
```

```
array([3298.31944421, 4865.52664198, 4889.62722658, ..., 3317.05674217,  
       3298.31944421, 3298.31944421])
```

- As we can see, the model can be directly used for making predictions without any retraining.

Deployment - Backend

Flask Web Framework

```
%%writefile backend_files/app.py

# Import necessary libraries
import numpy as np
import joblib # For loading the serialized model
import pandas as pd # For data manipulation
from flask import Flask, request, jsonify # For creating the Flask API

# Initialize Flask app with a name
superkart_api = Flask(__name__)

# Load the trained churn prediction model
model = joblib.load("tunedrandomforest_model.joblib")

# Define a route for the home page
@superkart_api.get('/')
def home():
    return "Welcome to the Superkart Sales Forecast Predication API home!"

# Define an endpoint to predict churn for a single customer
@superkart_api.post('/v1/predict')
def predict_sales():
    # Get JSON data from the request
    data = request.get_json()

    # Extract relevant customer features from the input data. The order of the columns
    sample = {
        'Product_Weight': data['Product_Weight'],
        'Product_Sugar_Content': data['Product_Sugar_Content'],
        'Product_Allocated_Area': data['Product_Allocated_Area'],
        'Product_MRP': data['Product_MRP'],
        'Store_Size': data['Store_Size'],
        'Store_Location_City_Type': data['Store_Location_City_Type'],
        'Store_Type': data['Store_Type'],
        'Product_Id_char': data['Product_Id_char'],
        'Store_Age_Years': data['Store_Age_Years'],
        'Product_Type_Category': data['Product_Type_Category']
    }

    # Convert the extracted data into a DataFrame
    input_data = pd.DataFrame([sample])
```

```

# Make a churn prediction using the trained model
prediction = model.predict(input_data).tolist()[0]

# Return the prediction as a JSON response
return jsonify({'Sales': prediction})

# Run the Flask app in debug mode
if __name__ == '__main__':
    superkart_api.run(debug=True)

```

Overwriting backend_files/app.py

```

# Observation :
# Creating the api endpoints @superkart_api.get('/') and @superkart_api.post('/v1/p
# The end point @superkart_api.post('/v1/predict') is used to predict the churn for

```

Dependencies File

```

%%writefile backend_files/requirements.txt
pandas==2.2.2
numpy==2.0.2
scikit-learn==1.6.1
seaborn==0.13.2
joblib==1.4.2
xgboost==2.1.4
joblib==1.4.2
Werkzeug==2.2.2
flask==2.2.2
gunicorn==20.1.0
requests==2.32.3
uvicorn[standard]
streamlit==1.43.2

```

Overwriting backend_files/requirements.txt

```

# Observation :
# Creating a requirements.txt file to add the library to run the application.

```

Dockerfile

```

%%writefile backend_files/Dockerfile
FROM python:3.9-slim

# Set the working directory inside the container
WORKDIR /app #Complete the code to mention the command in Docker to set the working

# Copy all files from the current directory to the containers working directory
COPY . .

# Install dependencies from the requirements file without using cache to reduce ima
RUN pip install --no-cache-dir --upgrade -r requirements.txt #Complete the code to

# Define the command to start the application using Gunicorn with 4 worker processe
# - '-w 4': Uses 4 worker processes for handling requests
# - '-b 0.0.0.0:7860': Binds the server to port 7860 on all network interfaces

```

```
# - `app:app`: Runs the Flask app (assuming `app.py` contains the Flask instance name)
CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:7860", "app:superkart_api"]
```

Overwriting backend_files/Dockerfile

```
# Observation :
# Creating a docker file containing ordered instructions (a blueprint or script) u
```

Setting up a Hugging Face Docker Space for the Backend

Note: We are creating a Hugging Face Docker Space for our backend using the Hugging Face Hub API. This automates the space creation process and enables seamless deployment of our Flask app.

```
from google.colab import userdata
from huggingface_hub import login
hf_token = userdata.get('HF_TOKEN').splitlines()[0]
login(token=hf_token)

from huggingface_hub import create_repo
```

```
# Try to create the repository for the Hugging Face Space
try:
    create_repo("SangeethaKumari/austinsuperkartpredictionbackend",
                repo_type="space",
                space_sdk="docker",
                private=False
    )
except Exception as e:
    # Handle potential errors during repository creation
    if "RepositoryAlreadyExistsError" in str(e):
        print("Repository already exists. Skipping creation.")
    else:
        print(f"Error creating repository: {e}")
```

Error creating repository: Client error '409 Conflict' for url 'https://huggingface.co/api/repos/create' (Request ID: Root=1-69fcc08a-59219a431ff7df2f54b93839;a30cc695-be1e-4a21-b14a-99619090b76a)

For more information check: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status/409>

You already created this space repo: SangeethaKumari/austinsuperkartpredictionbackend

```
# A backend space is created on hugging face at
# https://huggingface.co/spaces/SangeethaKumari/austinsuperkartpredictionbackend
# Accessing this Url we invokes the get api (@superkart_api.get('/')) and displays
```

Uploading Files to Hugging Face Space (Docker Space)

```
# for hugging face space authentication to upload files

from huggingface_hub import login
from google.colab import userdata
access_key = userdata.get('HF_TOKEN').splitlines()[0]
repo_id = "SangeethaKumari/austinsuperkartpredictionbackend"
```

```

# Login to Hugging Face platform with the access token
login(token=access_key)

# Initialize the API
api = HfApi()

# Upload Streamlit app files stored in the folder called deployment_files
api.upload_folder(
    folder_path="backend_files",
    repo_id=repo_id, # Hugging face space id
    repo_type="space", # Hugging face repo type "space"
)

```

No files have been modified since last commit. Skipping to prevent empty commit.
 WARNING:huggingface_hub.hf_api:No files have been modified since last commit. Skipping to prevent empty commit.

```

CommitInfo(commit_url='https://huggingface.co/spaces/SangeethaKumari/austinsuperkartpredictionbackend/commit/cd47d17c4c64d1252490c3053c15e4b8bf0e506e', commit_message='Upload folder using huggingface_hub', commit_description='', oid='cd47d17c4c64d1252490c3053c15e4b8bf0e506e', pr_url=None, repo_url=RepoUrl('https://huggingface.co/spaces/SangeethaKumari/austinsuperkartpredictionbackend', endpoint='https://huggingface.co', repo_type='space', repo_id='SangeethaKumari/austinsuperkartpredictionbackend'), pr_revision=None, pr_num=None)

```

```

# Observation :
# Here successful authentication is done on Hugging face session using the HF_TOKEN
# Successfully all the file under backend_files folder are uploaded on the hugging
# The model created using .joblib is also uploaded.
# After uploading the files, the Hugging Face infrastructure is triggered to build

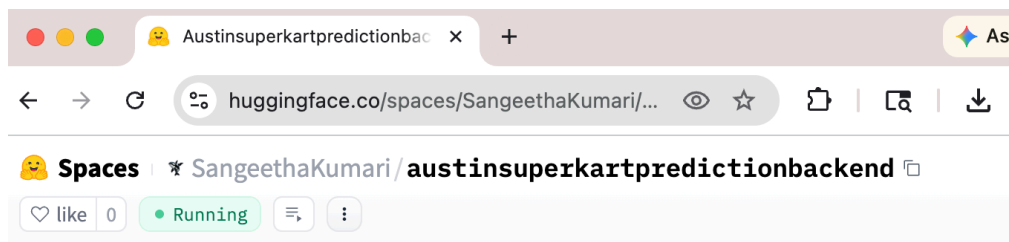
```

```

from IPython.display import Image

Image('backend.png', width=500)

```



Deployment - Frontend

Points to note before executing the below cells

- Create a Streamlit space on Hugging Face by following the instructions provided on the content page titled **Creating Spaces and Adding Secrets in Hugging Face** from Week 1

Streamlit for Interactive UI

```
# Create a folder for storing the files needed for frontend UI deployment
os.makedirs("frontend_files", exist_ok=True)
```

```
%%writefile frontend_files/app.py

import streamlit as st
import requests

st.title("Superkart Sales Prediction App")
```

```

# Input fields for product and store data
Product_Weight = st.number_input("Product Weight", min_value=0.0, value=12.66)
Product_Sugar_Content = st.selectbox("Product Sugar Content", ["Low Sugar", "Regular", "High Sugar"])
Product_Allocated_Area = st.number_input("Product Allocated Area", min_value=0.0, value=100.0)
Product_MRP = st.number_input("Product MRP", min_value=0.0, value=147.0)

# Categorical inputs based on your EDA findings
Store_Size = st.selectbox("Store Size", ["High", "Medium", "Small"])
Store_Location_City_Type = st.selectbox("Store Location City Type", ["Tier 1", "Tier 2", "Tier 3"])
Store_Type = st.selectbox("Store Type", ["Supermarket Type1", "Supermarket Type2", "Grocery Store"])

# Feature engineered inputs
Product_Id_char = st.selectbox("Product ID Category", ["FD", "DR", "NC"])
Store_Age_Years = st.number_input("Store Age (in Years)", min_value=0, value=20)
Product_Type_Category = st.selectbox("Product Type Category", ["Perishables", "Non-Perishables"])

product_data = {
    "Product_Weight": Product_Weight,
    "Product_Sugar_Content": Product_Sugar_Content,
    "Product_Allocated_Area": Product_Allocated_Area,
    "Product_MRP": Product_MRP,
    "Store_Size": Store_Size,
    "Store_Location_City_Type": Store_Location_City_Type,
    "Store_Type": Store_Type,
    "Product_Id_char": Product_Id_char,
    "Store_Age_Years": Store_Age_Years,
    "Product_Type_Category": Product_Type_Category
}

if st.button("Predict", type='primary'):
    response = requests.post("https://SangeethaKumari-austinsuperkartpredictionbackend.herokuapp.com/predict")
    if response.status_code == 200:
        result = response.json()
        predicted_sales = result["Sales"]
        st.write(f"Predicted Product Store Sales Total: ₹{predicted_sales:.2f}")
    else:
        st.error("Error in API request")

```

Writing frontend_files/app.py

```

#Observation :
# Creating a front end streamlit Interactive UI for accessing the backend endpoint

```

Dependencies File

```

%%writefile frontend_files/requirements.txt
requests==2.32.3
streamlit==1.45.0

```

Writing frontend_files/requirements.txt

DockerFile

```

%%writefile frontend_files/Dockerfile
# Use a minimal base image with Python 3.9 installed
FROM python:3.9-slim

```

```
# Set the working directory inside the container to /app
WORKDIR /app

# Copy all files from the current directory on the host to the container's /app dir
COPY . .

# Install Python dependencies listed in requirements.txt
RUN pip3 install -r requirements.txt

# Define the command to run the Streamlit app on port 7860 and make it accessible e
CMD ["streamlit", "run", "app.py", "--server.port=7860", "--server.address=0.0.0.0"]

# NOTE: Disable XSRF protection for easier external access in order to make batch p

Overwriting frontend_files/Dockerfile
```

```
# Try to create the repository for the Hugging Face Space for the front end
try:
    create_repo("SangeethaKumari/austinsuperkartpredictionfrontend",
                repo_type="space", # Specify the repository type as "space"
                space_sdk="docker", # Specify the space SDK as "docker"
                private=False # Set to True if you want the space to be private
    )
except Exception as e:
    # Handle potential errors during repository creation
    if "RepositoryAlreadyExistsError" in str(e):
        print("Repository already exists. Skipping creation.")
    else:
        print(f"Error creating repository: {e}")
```

```
# A frontend space is created on hugging face at
# https://huggingface.co/spaces/SangeethaKumari/austinsuperkartpredictionfrontend
# Accessing this Url , we get the streamlit interactive UI displaying the various i
```

Uploading Files to Hugging Face Space (Streamlit Space)

```
from huggingface_hub import login
access_key = userdata.get('HF_TOKEN').splitlines()[0]
print(access_key)
repo_id = "SangeethaKumari/austinsuperkartpredictionfrontend"

# Login to Hugging Face platform with the access token
login(token=access_key)

# Initialize the API
api = HfApi()

# Upload Streamlit app files stored in the folder called deployment_files
api.upload_folder(
    folder_path="frontend_files",
    repo_id=repo_id, # Hugging face space id
    repo_type="space", # Hugging face repo type "space"
)
```

hf_omcKeuqFloBsaAMfQuUNcTsZctIDaBPb0z

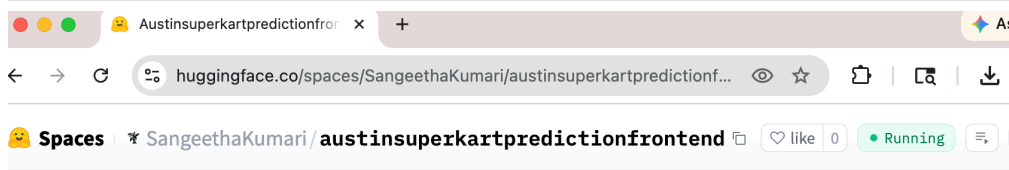
```
CommitInfo(commit_url='https://huggingface.co/spaces/SangeethaKumari/austinsuperkartpredictionfrontend/commit/2c4a0771ebdd98ab8df974fe078350fe8ccbd3', commit_message='Upload folder using huggingface_hub', commit_description='', oid='2c4a0771ebdd98ab8df974fe078350fe8ccbd3', pr_url=None, repo_url=RepoUrl('https://huggingface.co/spaces/SangeethaKumari/austinsuperkartpredictionfrontend', endpoint='https://huggingface.co', repo_type='space', repo_id='SangeethaKumari/austinsuperkartpredictionfrontend'), pr_revision=None, pr_num=None)
```

Observation :

*# Here successful authentication is done on Hugging face session using the HF_TOKEN
Successfully all the file under frontend_files folder are uploaded on the hugging
The Hugging Face url to access the frontend is https://huggingface.co/spaces/Sang
After uploading the files, the Hugging Face infrastructure is triggered to build*

```
from IPython.display import Image
```

```
Image('FrontEnd.png', width=500)
```



Superkart Sales Prediction App

Product Weight

12.66

- +

Product Sugar Content

Low Sugar

▼

Product Allocated Area

0.05

- +

Product MRP

147.00

- +

Store Size

High

▼

Store Location City Type

Tier 1

▼

Store Type

Supermarket Type1

▼

Product ID Category

FD

▼

```
from IPython.display import Image
```

```
Image('prediction.png', width=500)
```


Austinsuperkartprediction

huggingface.co/spaces/SangeethaKumari/austinsuperkartpredictionfrontend

Low Sugar

Product Allocated Area

0.05

Product MRP

147.00

Store Size

High

Store Location City Type

Tier 1

Store Type

Supermarket Type1

Product ID Category

FD

Store Age (in Years)

20

Product Type Category

Perishables

Predict

Predicted Product Store Sales Total: ₹4365.14

Actionable Insights and Business Recommendations

-Actionable Insights

- **Prioritize High-Revenue Categories:** The analysis shows that Fruits and Vegetables (14.3%) and Snack Foods (13.1%) are the primary revenue drivers.
- **Capitalize on Health Trends:** Over 50% of items sold are "Low Sugar."
- **Optimize Shelf Space:** The correlation matrix revealed that Product_Allocated_Area has a near-zero impact on sales.
- **Revitalize Underperforming Outlets:** OUT002 (Food Mart) in Tier 3 cities is significantly underperforming.
- **Pricing Consistency:** The analysis shows that Product_MRP is consistent across all store types

-Business Recommendation

- As mostly Fresh fruits and Vegetables are high revenue categories. Implement Just in time inventory system to avoid wastage
- Expanding the inventory for the "No Sugar" and "Low Sugar" segments, especially within the Snacks and Dairy categories, to provide customers with more healthier options.
- Stopping prioritizing large display areas for underperforming products. Instead, use that space for high-MRP items, which showed a strong positive correlation (0.79) with total sales.

- Maintain a unified pricing strategy to build brand trust, but consider localized "Bundle Discounts" in Tier 3 cities to increase the average transaction value.
- Use the Tuned Random Forest model ($R^2 = 0.6687$) to perform price-sensitivity analysis. By simulating changes in MRP for "Non-Perishable" goods, management can optimize pricing tiers to maximize total revenue.